KH 정보교육원

애플리케이션 설계

목 차

Part 1. 공통 모듈 설계하기

Chapter 1. 애플리케이션 설계 프로세스의 이해 4

Chapter 2. 애플리케이션 아키텍쳐 구축 가이드라인 설정 10

Chapter 3. 애플리케이션 공통 모듈 설계하기 27

Chapter 4. UML 작성 34

Part 2. 타 시스템 연동 설계하기

Chapter 5. MyBatis와 웹 애플리케이션 연동하기 52

Chapter 6. 오픈 API와 웹 애플리케이션 연동하기 69

Part 1. 공통 모듈 설계하기

Chapter 1. 애플리케이션 설계 프로세스의 이해

일반적인 소프트웨어 개발 프로세스는 요구사항 분석 -> 설계 -> 개발 -> 테스팅 의 단계로 나눌 수 있으며, 각 단계에 대응하는 검증 절차들을 가지게 됩니다.

소프트웨어 개발 프로세스에 중요한 과정 중에 하나로 '설계'라는 단계를 포함하고 있지만, 개발자 대부분은 설계문서 작성에 노력을 기울이지 않은 채 자신의 능력을 믿고 바로 코딩하는 방식을 선호하고 있습니다. 고객이 산출물로 설계서를 요구할 경우, 구현이 완료된 후에야 소스코드를 보고 설계문서가 작성되는 경우도 있습니다. 즉, 소프트웨어를 개발할 때 기본적으로 설계의 중요성을 인식하지 못하고 있는 실정입니다.

우리나라의 소프트웨어 개발 프로젝트 성공률이 20%라는 이야기를 들은 적이 있습니다.
(소프트웨어 개발 프로젝트의 성공은, 주어진 시간에 주어진 비용으로 요구된 품질의 제품을 만들어 내는 것으로 보고 있습니다.)
개발자들은 밤낮, 주말 가리지 않고 일해서 개발 종료날짜는 지켰어도 시간과 비용 면에서 성공적이지 못하고, 고객과 합의한 기능은 모두 구현 했지만, 고객이 만족하지 못해 성공으로 보기가 어려운 경우가 많습니다.
소프트웨어는 눈에 보이지 않기 때문에 일정과 비용을 예측하기가 쉽지 않고, 사용자가 실제로 사용해 보기 전에는 사용자의 의도를 파악하기 어려운 특징을 가지고 있습니다.

눈에 보이지 않는 소프트웨어를 가시화 하여 개발할 소프트웨어를 이해하고, 계획하는데 도움이 되는 작업 중에 하나가 설계 작업 입니다.

여기서 잠깐! 소프트웨어 설계는 무엇인지 보도록 하겠습니다.

『 Software design is a process of problem solving and planning for a software solution. After the purpose and specifications of software are determined, software developers will design or employ designers to develop a plan for a solution. It includes low-level component and algorithm implementation issues as well as the architectural view.
소프트웨어 설계는 소프트웨어 해결책을 위한 문제 해결과 계획 과정이다. 소프트웨어의 목적과 명세가 결정되면 개발자가 설계 하거나 설계자를 고용하여 해결책을 위한 계획을 개발하도록 한다. low-level component 와 알고리즘 구현 문제, 그리고 구조에 대한 architectural view 가 포함된다. 』

『 본격적인 프로그램의 구현에 들어가기 전에 소프트웨어를 구성하는 뼈대를 정의해 구현의 기반을 만드는 것 』

소프트웨어 개발은 고객(도메인)의 문제(요구사항)를 이해하고, 문제를 소프트웨어로 해결(구현)하고자 하는 노력으로 볼 수 있습니다.

사실, 간단한 문제야 이해와 계획 없이도 바로 해결이 가능합니다. 또는 경험이 많고 유능한 해결사가 나타나 짠 하고 한방에 문제를 해결하기도 합니다. 하지만, 단순한 문제가 아닌 시간이 오래 걸리는 복잡하고 거대한 문제라면? 문제 해결에 참여하는 사람이 한두 명이 아닌 수 천명이라면? 여러 문제가 복잡하게 엉켜 있어 문제간의 연결 고리를 찾아 연결시켜줘야 한다면? 유능한 해결사가 떠나고 난 뒤, 지금 문제와 비슷한 형태의 문제를 해결해야 할 때가 온다면? 많은 조건과 어려움 속에서 문제를 성공적으로 잘 해결하기 위해선 우리가 해결해야 할 문제가 무엇인지 이해하고, 문제 해결을 위한 해결책을 잘 계획 해야 하는 작업이 반드시 필요 할 것입니다.

아키텍처 요구사항 및 제약사항 등의 아키텍처 영향요소를 고려하여 아키텍처 설계 전략을 수립하고 아키텍처를 설계하는데 목적이 있습니다.

1. 테일러링가이드(Tailoring Guide)

1. 아키텍처를 재정의 하지 않고 기존의 아키텍처를 그대로 활용하는 경우에는 아키텍처설계 액티비티를 생략할 수 있다.
2. 아키텍처가 복잡하지 않고 규모가 작은 경우에는 아키텍처 도메인(SW 아키텍처, 데이터아키텍처, 기술아키텍처) 별로 산출물을 작성하지 않고 하나로 통합하여 작성할 수 있다.

2. 고려사항(Considerations)

- SW 아키텍처, 데이터 아키텍처, 기술 아키텍처에 대한 설계는 동시에 진행되도록 한다.
- SA, TA, DA 가 각자의 아키텍처 도메인별로 설계를 하지만 상호 정합성 및 보완사항에 대한 확인 작업을 통하여 일관성을 유지하도록 한다.

SW 아키텍처 구조를 정의하고 구성요소간의 상호작용을 분석하여 구성요소간의 관계와 구성요소의 역할과 책임에 대해 정의하고 상세 설계하는데 목적이 있습니다.

1. 수행절차

1. 소프트웨어 아키텍처의 범위를 구체화한다.
   - 선행작업에서 작성된 아키텍처 분석서, 아키텍처 이슈 목록, 솔루션 및 재사용 자산 분석서 등을 참조하여, 소프트웨어 아키텍처에서 제공해야 하는 설계 및 개발 지침을 정의하기 위해 고려되어야 하는 업무 범위와 솔루션 범위를 파악한다.
2. SW 기술 유형을 정의한다.
   - 아키텍처 영향 요소 및 해당 기능을 달성하는데 활용되는 솔루션 군을 중심으로 품질속성에 따라 해당 프로젝트에 필요한 SW 기술 유형을 나누고 해당 기술 유형에 대한 기술 구조를 정의한다. (예, 웹 기술 유형, 배치 기술 유형, DW 기술 유형 등)
   - 해당 기술 유형별로 필요한 구성 요소를 도출하고 이들간의 관계를 표현한다. (일반적으로 소스코드 구성관계를 표현하는 Layer 뷰와 런 타임 시에 주요 기술 구성요소와 관계를 표현하는 Run-time 뷰, 어플리케이션이 배치되는 Delpolyment 뷰를 기본적으로 사용할 수 있다.)
   - 정보 시스템간의 연계를 수행하기 위한 연계 아키텍처를 정의하고 연계 대상 시스템간의 연계 구성을 도식화하고 연계 범위를 표현한다. (연계 구성 및 범위를 도식화하는데 있어서 연계 논리모델 다이어그램을 작성할 수 있다.)
3. SW 기술 구조 별 설계 및 개발 지침을 작성한다.
   - SW 기술 유형이 정의되면, 각 SW 기술 구조 별로 설계 및 개발 지침을 작성한다. 소프트웨어 아키텍처 설계 전략 검토서에서 의사 결정한 결과와 핵심 근거 및 SW 아키텍처 설계 단계에서 의사 결정한 결과와 핵심 근거를 기술한다.
   - 상기 의사 결정 사항 중에서 별도의 시스템 공통 모듈을 개발을 해야 하는 경우 그 규모와 양을 고려해서 개발표준 구현 단계에서 수행하도록 한다.
   - 상기 의사 결정 사항 중에서 정보 시스템간 연계 기능 구현을 위하여 연계 공통기능, 연계 유형(패턴)을 개발해야 하는 경우 그 규모와 영을 고려해서 개발표준 구현 단계에서 수행하도록 한다.
4. SW 아키텍처의 적용 현황을 파악한다.
   - 관련 시스템 별로 설계 의사결정을 한 결과를 시스템 전체를 보여주는 Context 레벨의 다이어그램 형태에서 목표 시스템의 구성과 해당 구성요소간의 상호작용 관계를 다양한 관점에서 표현될 수 있다. (시스템 구성 측면, 시스템간 연계 측면, 시스템 별 적용 솔루션 배치 측면 등)

- 정의된 SW 아키텍처에 대해 각 업무에서 적용이 된 모습을 다양한 관점에서 표시하여, 해당 어플리케이션 별로 아키텍처가 반영되었음을 표현한다.
- 상기 활동은 아키텍처정의서를 완료한 이후에 보완을 할 수 있다.

5. PL 은 타 아키텍처와의 상호 정합성을 검토한다.

2. 수행가이드

1. 목표시스템의 규모가 큰 경우, 한 개 이상의 소프트웨어 기술구조가 정의될 수 있으며, 해당 기술 구조 유형이 적용되는 시스템의 범위가 명시되어야 한다.
2. SW 아키텍처 측면에서는 모듈구조, 런타임 구성, 배포 구조, 연계 구조 관점에서 표현할 수 있다.
3. 시스템 전체 측면에서의 아키텍처 상관관계를 이해하기 위해서 목표시스템 레벨의 다양한 관점을 프로젝트 상황에 따라 추가하여 작성할 수 있다. 이 경우 동일한 이해관계자의 고려사항을 반영이 반영될 수 있도록 일관성 있게 정리되어야 한다.
4. SW 아키텍처 정의서는 기본적으로 SW 아키텍처의 의사결정 사항이 기술되므로 설계 산출물이 작성되기 전에 주요 설계 의사결정 사항들이 정리될 수 있도록 분석 단계가 완료되는 시점에 완료 되어야 한다. 하지만 아키텍처정의서는 설계를 하면서 도출되는 이슈를 반영해야 하므로 아키텍처 정의서의 완료는 설계 단계 완료단계까지 이어짐을 고려한다.

아키텍처 이슈항목 및 아키텍처 설계과정에서 발생하는 이슈항목에 대해서 이슈를 해결하기 위한 대안을 도출하고 대안분석을 통해 아키텍처 설계전략을 정의합니다.

1. 수행절차

1. SA 는 아키텍처 분석 및 솔루션 및 재사용 자산 분석에서 도출된 아키텍처 이슈에 대해서 설계전략 검토 대상을 선정한다.
   - 해당 아키텍처 이슈에 대해서 내부적으로 해결방안이 명확하거나 또는 기 도출된 설계전략을 참조할 수 있다면 생략이 가능하며 이 경우 아키텍처 정의서에 해당 내용과 선정 근거에 대해서 기술한다.
2. 각 아키텍처 이슈에 대한 설계 전략 후보를 검증하고 의사결정 사항을 도출한다.
   - SA 는 아키텍처 이슈에 대한 설계 대안을 도출한다.
   - SA 는 설계 대안을 분석하며 수행하는데 필요한 자원과 전제사항을 도출한다.
   - 설계 대안에 대한 Trade-off 를 분석하여 장단점을 정리한다.

3. SA 는 선정된 설계 전략들 간에 충돌이 없는지 검토하고 아키텍처 이슈에 대해 모두 검토 되었는지 확인한다.
4. SA 는 기술 및 데이터 아키텍처와의 정합성을 검토한다.

2. 수행가이드

1. 아키텍처 이슈에 대해서 아키텍트의 수행경험, Best Practice 등을 참고하여 설계전략 후보를 도출하되 기술적 위험이 큰 경우에 대해서 설계전략 검토서의 항목으로 작성을 하도록 한다. 이를 통해서 결정된 의사결정에 대해서 아키텍처정의서에 기술한다.

- 기술적 난이도는 낮지만 아키텍처적으로 중요한 설계전략은 바로 아키텍처정의서에 기술한다.
  - 설계전략 검토서 주요 작성항목
  - 아키텍처 설계 전략 정의 대상 : 아키텍처 분석에서 도출된 아키텍처 이슈에 대한 설계전략 수립 대상을 선정하여 기술한다.
    - 설계 전략 대안 검토 : 설계 전략 수립대상 별로 검토한 내용을 기술한다. 설계 전략은 아키텍처 스타일, 컴포넌트 도출 전략, 배치작업 전략, 트랜잭션 처리 전략, 통합 로깅 전략, 사용자 인증 전략, Legacy 연계 전략 등과 같이 반드시 대안이 고려되어야 하는 시스템 전체 품질속성에 영향을 줄 수 있는 항목을 대상으로 작성하도록 한다.
    - 설계 전략 개요 : 설계 전략의 이슈 및 수립된 전략에 대해 간단하게 기술한다.
    - 설계 전략 (내용) : 수립된 전략의 구체적인 내용, 즉 설계대안에 대한 장단점을 기술한다

  기술 위험이 큰 설계전략의 경우는 가급적 PoC 를 수행하여 의사결정 사항을 명확하게 해야 한다.

시스템을 구축하기 위한 기술아키텍처에 대한 설계 원칙을 정의하고 구성요소를 도출하며 구성요소 별로 설계를 수행하는데 목적이 있습니다.

1. 수행절차

1. 하드웨어 아키텍처를 품질속성을 고려하여 설계한다.
   - 서버별로 H/W, OS, 탑재되는 시스템 소프트웨어를 중심으로 기능 및 구성을 설계한다.
   - 스토리지를 사용하는 시스템 및 데이터를 정의하고 스토리지 구성을 설계한다.
   - 기타 구성요소에 대한 구성을 설계한다.
   - 용량산정이 필요한 경우 수행한다.
2. 네트워크 아키텍처를 품질속성을 고려하여 설계한다.
   - 용량산정이 필요한 경우 수행한다.
3. 시스템 소프트웨어를 품질속성을 고려하여 설계한다.
4. 데이터베이스를 품질속성을 고려하여 설계한다.

Chapter 2. 애플리케이션 아키텍쳐 구축 가이드라인 설정

1. 적용범위

본 가이드라인은 차량에 적용되는 전장품(Electronics Control Unit)에 대해 기능안전 적용을 위한 소프트웨어 개발 절차를 정의한다.

2. 목적

본 가이드라인의 목적은 차량에 장착되는 전장품에 잠재된 오류 없이 설계된 목적으로 동작할 수 있도록 협력업체가 자동차 기능안전(ISO 26262)를 고려한 고품질의 소프트웨어를 개발할 수 있도록 돕기 위함이다.

3. 용어의 정의

검증(Verification)

개발자 혹은 시험자의 기능 활용에 있어 소프트웨어가 명시된 요구들을 충족시키는 여부를 판단하기

위한 활동

확인(Validation)

사용자 요구 또는 소프트웨어 요구에 적합한지를 입증하기 위한 활동

정적 분석(Static Analysis)

프로그램 코드(시험용 소프트웨어)를 직접 수행시키지 않고 수작업 또는 기계로 코드에 내장된

오류를 발견하는 시험

동적 분석(Dynamic Analysis)

프로그램 코드를 직접 수행시켜 오류를 발견하는 시험

블랙 박스 시험(Black Box Test)

프로그램의 내부 구조 및 특성에 대한 정보 없이 설계 내용으로부터 직접 테스트 항목 정리하는 방법

화이트 박스 시험(White Box Test)

프로그램의 내부 논리구조를 바탕으로 프로그램 코드를 분석하여 Test Case 를 설계하는 시험

시험 사례(Test Case)

시험하는 사람을 위해 문서화된 지침으로 한 기능이나 여러 기능의 조합이 어떻게 시험되어야

하는가를 명시한다. (시험목적, 기능, 시험환경 및 조건, 시험 데이터, 절차)

코드 커버리지 분석(Code Coverage Analysis)

시험하는 소스 코드에 대해 조합 가능한 모든 코드 패스를 파악해서 실행 중에 어떠한 코드 패스가

실행되었는지를 확인하고 이를 명시한다.

※ 기타 다른 용어는 용어집 또는 표준 KS R ISO 26262-1, ISO 26262-1 을 참조한다.

4. 일반사항

다음은 본 가이드라인의 이해를 돕기 위한 설명 사항이다.

산출물 관련

- 제시된 산출물은 기능안전 표준(즉, ISO 26262)에서 제시하는 기능안전 표준 산출물을 의미한다.
- 아이템에 부여된 ASIL 등급에 따라 필요 산출물이 달라 질 수 있다.
- 산출물이 표준 산출물명과 반드시 일치할 필요는 없지만, 해당 정보를 확인 가능해야 한다.
- 각 단계의 산출물에 대한 책임과 권한은 R:작성, A:승인, S:지원, I:통보로 표시된다. 이 R&R은 참고 사항이다.
- 제출 형태는 Full document(전체 문서), Summary(중요정보 요약), 필요 시 on-site 확인으로 구분된다.
- 산출물의 작성여부, R&R, 제출형태, 사항은 DIA(Development Interface Agreement) 작성 시점에 구체화 된다.

(예시) 산출물 표

산출물명	QM	A	B	C	D	R:작성	A:승인	S:지원	I:통보	제출형태
[1] Development Interface Agreement	○	○	○	○	○	OEM			Supplier	Full document

표 해석

- 표는 기능안전 표준(즉, ISO 26262)에서 제시하고 해당 주제에 대한 수행 방법 및 갖추어야 할 속성을 표현한 것이다. (예시의 표는 "하드웨어 설계 검증"을 수행하기 위한 방법들을 제시한다.)

- 방법 및 속성은 연속항목(예를 들어 1,2,3) 또는 대안항목(예를 들어 3a,3b,3c)으로 구분된다.
- 연속항목의 경우 연속되는 각 항목이 ASIL에 따라 권장하는 대로 적용 한다.
- 대안항목의 경우 각 ASIL에 지정된 방법들을 적절히 조합하여 적용 한다.
- 선택되어 수행되는 방법에 대한 적절한 근거가 제시되어야 한다.
- (예시) ASIL C가 부여된 아이템 개발에 있어 하드웨어 설계를 검증하기 위한 방법은 다음과 같이 선택할 수 있다. 연속항목(1,2,3)은 각각 선택되어야 한다. 대안항목(1a,1b)는 ASIL 수준에 따라 매우 권장(++)되는 사항을 선택할 수 있다. 따라서, {1b,2,3a} 또는 {1b,2,3b}과 같이 선택할 수 있다.
-

(예시) 하드웨어 설계 검증

방법 또는 속성		ASIL			
		A	B	C	D
1a	하드웨어 설계에 대한 워크스루(walk-through)	++	++	o	o
1b	하드웨어 설계에 대한 인스펙션(inspection)	+	+	++	++
2	안전 분석	**안전 분석 내용 참고**			
3a	시뮬레이션	o	+	+	+
3b	하드웨어 시작품(prototyping)에 의한 개발	o	+	+	+
++ : 매우 권장, + : 권장, o : 권장사항 없음					

5. 소프트웨어 개발 (Software Development)

본 절은 기능안전 표준(ISO 26262)에서 제시하고 있는 소프트웨어 수준의 제품 개발의 활동(process), 산출물(work product)에 대해 다룬다.

소프트웨어 수준의 제품 개발이란 협력사가 시스템 개발의 단계에서 작성한 기술안전 개념(Technical Safety Concept)과 시스템 사양서를 바탕으로 구체화하여 소프트웨어를 개발하는 것을 의미한다. 본 가이드라인에서는 기능안전 활동만을 고려하고, 기능안전 외의 개발 절차는 해당 협력사의 내부 개발 절차를 따른다.

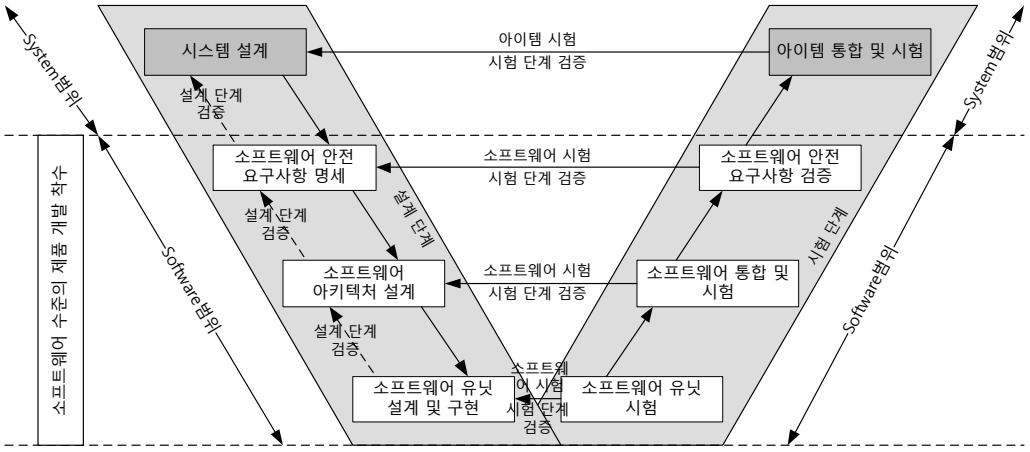
기능안전 표준(ISO 26262)에서는 소프트웨어 수준에서의 제품 개발 활동을 다음과 같이 정의하고 있다. 각 활동의 세부 내용은 하위 절에 기술한다.

- 1단계: 소프트웨어 수준의 제품 개발 착수
- 2단계: 소프트웨어 안전 요구사항의 명세
- 3단계: 소프트웨어 아키텍처 설계
- 4단계: 소프트웨어 단위 설계 및 구현

- 5단계: 소프트웨어 단위 시험
- 6단계: 소프트웨어 통합 및 시험
- 7단계: 소프트웨어 안전요구사항 검증

이외에 다음 내용에 대해 기술한다.

- 소프트웨어 도구 사용의 신뢰 확보
- 소프트웨어 컴포넌트 인정



소프트웨어 수준의 제품 개발을 위한 ISO 26262 참조 단계 모델

소프트웨어 수준의 제품 개발 착수

Attribute	Description
목표 (Objectives)	• 소프트웨어 개발 하위 단계 동안 수행될 기능안전 활동을 결정하고 계획하고 착수
입력물 (Prerequisite)	• 전체 프로젝트 계획 • 안전 계획 • 기술안전 개념 • 시스템 설계 명세서 • 아이템 통합 및 시험 계획
활동 (Activities)	• 소프트웨어 수준의 기능안전 활동을 정의

산출물 (Work product)										
산출물명	**QM**	**A**	**B**	**C**	**D**	**R:작성**	**A:승인**	**S:지원**	**I:통보**	**제출형태**

[3] Safety plan (supplier activities)		○	○	○	○	Supplier	OEM			Full document
[52] Design and coding guidelines for modelling and programming languages		○	○	○	○	Supplier				필요시 on-site 확인
[53] Tool application guidelines		○	○	○	○	Supplier				필요시 on-site 확인
[54] Software verification plan	○	○	○	○	○	Supplier				Full document
활동 요구사항 (Requirement of Activities)										

5.1.1 소프트웨어 수준의 기능안전 활동 정의

1) 활동 및 소프트웨어 수준의 제품 개발에 적합한 방법이 결정되어야 한다.

2) 필요 시, 소프트웨어 수준의 각 활동을 조정(tailoring)해야 한다.

3) 수명주기 단계, 방법, 언어, 도구를 포함하여 한 아이템의 소프트웨어를 위한 소프트웨어 개발 프로세스는 소프트웨어 수명주기의 모든 하위단계에 걸쳐서 일정해야 한다.

4) 소프트웨어 개발의 모든 하위단계의 경우 적용 지침을 포함하여 다음 사항을 선택해야 한다.

a) 방법

b) 해당되는 도구

5) 적합한 모델이나 프로그래밍 언어를 선택 할 때 고려해야 할 기준은 다음과 같다.

a) 모호하지 않은 정의

b) 임베디드 실시간 소프트웨어와 런타임 오류처리에 대한 지원

c) 모듈화, 추상화 및 구조화된 구성을 위한 지원

6) 설계 및 구현의 정확성을 뒷받침하기 위하여 모델링이나 프로그래밍 언어에 대한 설계 및 코딩 지침은 아래 표에 나열된 주제를 다루어야 한다.

속성	ASIL			
	A	B	C	
낮은 복잡도 강제	++	++	++	
언어의 서브셋 사용	++	++	++	
강한 타입 체크 강제	++	++	++	
방어적 구현 기법 사용	o	+	++	
입증된 설계 원리 사용	+	+	+	
모호하지 않은 그래픽 표현 사용	+	++	++	
스타일 지침 사용	+	++	++	
명명 규칙 사용	++	++	++	

매우 권장, + : 권장, o : 권장사항 없음

※ "기능안전 적용을 위한 기능안전관리(FSM)/시스템 개발 가이드라인"의 부속서 A를 참고하여 안전 계획서 상의 소프트웨어 수준의 기능안전 활동을 갱신한다.

참고 표준 (Reference)	• ISO 26262-6, 5절 소프트웨어 수준의 제품 개발 착수 (Initiation of product development at the software level)

소프트웨어 안전 요구사항의 명세

Attribute	Description
목표 (Objectives)	• 소프트웨어 안전 요구사항을 명시 • 하드웨어와 소프트웨어간의 인터페이스 요구사항 상세화 • 소프트웨어 안전 요구사항과 하드웨어와 소프트웨어간의 인터페이스 요구사항이 기술안전 개념과 시스템 설계 명세서와 일치함을 검증
입력물 (Prerequisite)	• 기술안전 개념 • 시스템 설계 명세서 • 하드워어와 소프트웨어간 인터페이스 명세서

	• 안전 계획 • 소프트웨어 검증 계획
활동 (Activities)	• 하드웨어의 제약들과 이런 제약들이 소프트웨어에 미치는 영향을 고려한 소프트웨어 안전 요구사항 명세서 작성

산출물 (Work product)

산출물명	QM	A	B	C	D	R:작성	A:승인	S:지원	I:통보	제출형태
[54] Software verification plan		○	○	○	○	Supplier	OEM			Full document
[55] Software verification specification	○	○	○	○	○	OEM			Supplier	Full document
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document
[57] Software safety requirements specification		○	○	○	○	Supplier				필요시 on-site 확인
[30] Hardware-software interface specification		○	○	○	○	Supplier	OEM			Full document

활동 요구사항 (Requirement of Activities)

5.2.1 소프트웨어 안전 요구사항 명세서 작성

1) 소프트웨어 안전 요구사항은 소프트웨어 기반의 기능 고장으로 인해 소프트웨어에 할당된 기술안전 요구사항의 위반을 야기할 수 있는 모든 소프트웨어 기반의 기능을 다루어야 한다.

2) 기술안전 개념과 시스템 설계서로부터 생성되어야 하며 다음 사항을 고려해야 한다.

a) 안전 요구사항의 명세 및 관리
b) 명세된 시스템과 하드웨어 설정
c) 하드웨어와 스프트웨어간 인터페이스 명세서
d) 하드웨어 설계 명세서와 관련된 요구사항
e) 시간제약

f) 외부인터페이스

g) 소프트웨어에 영향을 미치는 차량, 시스템 또는 하드웨어의 작동모드

3) ASIL 분해가 소프트웨어 안전 요구사항에 적용되면 KS R ISO 26262-9:2012, 5절을 준수해야 한다.

4) 하드웨어와 소프트웨어간 인터페이스 명세서는 하드웨어의 올바른 제어 및 사용이 가능한 수준으로 자세하게 설명되어 있어야 하고 각각의 하드웨어와 소프트웨어사이의 안전과 관련된 종속 관계가 기술되어야 한다.

5) 안전 요구사항에 추가로 임베디드 소프트웨어에서 수행되는 그 외의 안전 요구사항을 위한 다른 기능들이 존재한다면 이런 기능들을 명시하거나 그 명세에 대한 참조가 있어야 한다.

5.2.2 소프트웨어 안전 요구사항 검증(Verification)

6) 소프트웨어 안전 요구사항, 하드웨어와 소프트웨어간 인터페이스의 갱신된 명세서에 대한 검증은 KS R ISO 26262-8:2012, 9절에 따라 계획되어야 한다.

7) 갱신된 하드웨어와 소프트웨어간 인터페이스 명세서는 시스템, 하드웨어, 소프트웨어 개발 담당자가 공동으로 검증해야 한다.

8) 소프트웨어 안전 요구사항과 갱신된 하드웨어와 소프트웨어간 인터페이스 요구사항은 다음 사항을 보여주기 위해 검증해야 한다.

a) 기술안전 요구사항과의 부합 및 일치

b) 시스템 설계와의 부합

c) 하드웨어와 소프트웨어간 인터페이스와의 일치

참고 표준 (Reference)	• ISO 26262-6, 6절 소프트웨어 안전 요구사항 명세 (Specification of software safety requirements)

소프트웨어 아키텍처 설계

Attribute	Description
목표 (Objectives)	• 소프트웨어 안전 요구사항을 구현하는 소프트웨어 아키텍처 설계를 개발 • 소프트웨어 아키텍처 설계를 검증
입력물 (Prerequisite)	• 안전 계획 • 모델링과 프로그래밍 언어를 위한 설계 및 코딩 지침 • 하드웨어와 소프트웨어간 인터페이스 명세서 • 소프트웨어 안전 요구사항 명세서

	• 소프트웨어 검증 계획 • 소프트웨어 검증 보고서
활동 (Activities)	• 소프트웨어 아키텍처 설계

산출물 (Work product)										
산출물명	**QM**	**A**	**B**	**C**	**D**	**R:작성**	**A:승인**	**S:지원**	**I:통보**	**제출형태**
[3] Safety plan (supplier activities)		○	○	○	○	Supplier	OEM			Full document
[57] Software safety requirements specification		○	○	○	○	Supplier				필요시 on-site 확인
[58] Software architectural design specification		○	○	○	○	Supplier				필요시 on-site 확인
[59] Safety analysis report (dependent failures analysis report 포함)		○	○	○	○	Supplier				필요시 on-site 확인
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document

활동 요구사항 (Requirement of Activities)

5.3.1 소프트웨어 아키텍처 설계

1) 소프트웨어 아키텍처 설계는 올바르고 효과적으로 이후의 개발 활동을 수행하는데 필요한 정보를 포착하기 위해서 아래에 나열된 소프트웨어 아키텍처 설계를 위한 표기법을 이용하여 적합한 추상화 수준으로 기술되어야 한다.

방법	**ASIL**			
	A	**B**	**C**	
비정형 표기법	++	++	+	

1b	준정형 표기법	+	++	++	++
1c	정형 표기법	+	+	+	+
++ : 매우 권장, + : 권장, o : 권장사항 없음					

2) 소프트웨어 아키텍처 설계를 개발하는 동안 다음 사항을 고려해야 한다.

a) 소프트웨어 아키텍처 설계의 검증 가능성

b) 설정 가능한 소프트웨어의 적합성

c) 소프트웨어 단위의 설계 및 구현의 타당성

d) 소프트웨어 통합 시험 동안의 소프트웨어 아키텍처에 대한 시험 가능성

e) 소프트웨어 아키텍터 설계의 유지 보수성

3) 높은 복잡도로 인한 고장을 피하기 위해 소프트웨어 아키텍처 설계는 아래 표에 나열된 원칙을 이용하여 다음과 같은 속성을 나타내야 한다.

a) 모듈성

b) 캡슐화

c) 단순성

방법	ASIL		
	A	B	C
소프트웨어 컴포넌트의 계층적 구조	++	++	++
소프트웨어 컴포넌트의 제한된 크기	++	++	++
인터페이스의 제한된 크기	+	+	+
각 소프트웨어 컴포넌트 내 높은 응집도(cohesion)	+	++	++
소프트웨어 컴포넌트 사이 제한된 결합도(coupling)	+	++	++
적합한 스케줄링 속성	++	++	++
인터럽트의 제한된 사용	+	+	+
매우 권장, + : 권장, o : 권장사항 없음			

4) 소프트웨어 아키텍처 설계는 모든 소프트웨어 단위가 식별되는 수준으로 개발되어야 한다.

5) 소프트웨어 아키텍처 설계는 다음 사항을 설명해야 한다.

a) 소프트웨어 컴포넌트의 정적 설계 측면

b) 소프트웨어 컴포넌트의 동적 설계 측면

6) 모든 안전관련 소프트웨어 컴포넌트는 다음 중 하나로 분류되어야 한다.
   a) 신규 개발
   b) 수정(modification)하여 재사용
   c) 수정(modification)없이 재사용

7) 새롭게 개발하거나 수정(modification)하여 재사용되는 안전관련 소프트웨어 컴포넌트는 KS R ISO 26262에 따라서 개발되어야 한다.

8) 수정(modification) 없이 재사용되는 안전관련 소프트웨어 컴포넌트는 KS R ISO 26262-8:2012, 12절에 따라서 사용 자격을 획득해야 한다.

참고 표준 (Reference)	• ISO 26262-6, 7절 소프트웨어 아키텍처 설계 (Software architectural design)

소프트웨어 단위 설계 및 구현

Attribute	**Description**
목표 (Objectives)	• 소프트웨어 아키텍처 설계 및 관련된 소프트웨어 안전 요구사항에 따라 소프트웨어 단위 명세 • 명세 되어있는 대로 소프트웨어 단위를 구현 • 소프트웨어 단위 설계 및 구현에 대한 정적 검증을 수행
입력물 (Prerequisite)	• 모델링 및 프로그래밍 언어를 위한 설계 및 코딩 지침 • 소프트웨어 검증 계획 • 소프트웨어 아키텍처 설계 명세서 • 안전 계획 • 소프트웨어 안전 요구사항 명세서 • 소프트웨어 검증 보고서
활동 (Activities)	• 소프트웨어 아키텍처 설계를 기반으로 소프트웨어 단위의 상세 설계 진행 • 소프트웨어 단위 설계 검증

산출물 (Work product)										
산출물명	**QM**	**A**	**B**	**C**	**D**	**R:작성**	**A:승인**	**S:지원**	**I:통보**	**제출형태**
[60] Software unit design specification		○	○	○	○	Supplier	OEM			Full document

[61] Embedded software	○	○	○	○	○	Supplier			OEM	Full document
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document
[62] Configuration data specification (OEM specific region only)	○	○	○	○	○	Supplier			OEM	Summary
[63] Calibration data specification (OEM specific region only)	○	○	○	○	○	Supplier			OEM	Summary
활동 요구사항 (Requirement of Activities)										

5.4.1 소프트웨어 단위 설계

1) 소프트웨어 단위 설계는 다음 개발 활동이 올바르고 효과적으로 수행되는데 필요한 정보를갖고 있음을 보장하기 위해 다음에 나열된 표기법을 사용하여 기술해야 한다.

방법	ASIL			
	A	B	C	
자연 언어	++	++	++	
비정형 표기법	++	++	+	
준정형 표기법	+	++	++	
정형 표기법	+	+	+	

매우 권장, + : 권장, o : 권장사항 없음

2) 소프트웨어 단위의 명세서는 구현을 위한 충분한 상세 수준으로 기능적 행동과 내부 설계를 설명해야 한다.

3) 아래 나열된 소스 코드 수준에서 소프트웨어 단위 설계 및 구현을 위한 설계 원리는 다음과 같은 속성을 달성하기 위해 적용되어야 한다.

a) 소프트웨어 아키텍처 설계에 기반한 소프트웨어 단위 내 서브프로그램과 기능의 올바른 실행 순서
b) 소프트웨어 단위 사이 인터페이스의 일관성
c) 소프트웨어 단위 사이 및 단위 내에서의 데이터 흐름과 제어 흐름의 정확성
d) 단순성
e) 가독성 및 이해 가능성
f) 강건성

방법	ASIL		
	A	B	C
서브프로그램과 함수에서 하나의 진입점과 하나의 종료점	++	++	++
동적 개체 또는 변수를 사용하지 않음, 혹은 그렇지 않으면 동적 변수 생성시 온라인 시험	+	++	++
변수 초기화	++	++	++
변수 이름을 다목적으로 사용하지 않음	+	++	++
전역 변수를 사용하지 않음, 만약 사용해야 한다면 그 사용에 대해 정당화한다.	+	+	++
포인터의 제한된 사용	o	+	+
암시적 형 변환 없음	+	++	++
숨겨진 데이터 흐름이나 제어 흐름 없음	+	++	++
무조건적 점프 없음	++	++	++
재귀 없음	+	+	++

					+
++ : 매우 권장, + : 권장, o : 권장사항 없음					

5.4.2 소프트웨어 단위 설계 검증

1) 소프트웨어 단위 설계 및 구현은 다음 사항을 증명하기 위해 아래 표에 나열된 검증 방법을 적용하여 검증되어야 한다.

a) 하드웨어와 소프트웨어간 인터페이스 명세와의 부합
b) 추적성을 통해 소프트웨어 단위에 할당된 소프트웨어 안전 요구사항의 충족
c) 설계 명세서와 소스 코드와의 부합

방법	ASIL			
	A	B	C	
워크스루(Walk-through)	++	+	o	
인스펙션(Inspection)	+	++	++	
준정형 검증	+	+	++	
정형 검증	o	o	+	
제어 흐름 분석	+	+	++	
데이터 흐름 분석	+	+	++	
정적 코드 분석	+	++	++	
의미적 코드 분석(Semantic code analysis)	+	+	+	

매우 권장, + : 권장, o : 권장사항 없음

참고 표준 (Reference)	• ISO 26262-6, 8절 소프트웨어 단위 설계 및 구현 (Software unit design and implementation)

소프트웨어 단위 시험

Attribute	Description
목표 (Objectives)	• 소프트웨어 단위가 소프트웨어 단위 설계 명세서를 충족하며 원하지 않는 기능은 포함하지 않는다는 것을 증명
입력물 (Prerequisite)	• 하드웨어와 소프트웨어간 인터페이스 명세서 • 소프트웨어 검증 계획

	• 안전 계획 • 소프트웨어 단위 설계 명세서 • 소프트웨어 단위 구현 • 소프트웨어 검증 보고서
활동 (Activities)	• 소프트웨어 단위 설계 명세서에 대한 소프트웨어 단위를 시험하는 절차가 수립되고 이 절차에 따라 시험이 실시

산출물 (Work product)										
산출물명	**QM**	**A**	**B**	**C**	**D**	**R:작성**	**A:승인**	**S:지원**	**I:통보**	**제출형태**
[54] Software verification plan	○	○	○	○	○	Supplier				Full document
[55] Software verification specification	○	○	○	○	○	OEM			Supplier	Full document
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document
활동 요구사항 (Requirement of Activities)										

- 다음은 ISO 26262 표준에서 제시하고 있는 소프트웨어 단위 시험 방법이다.

5.5.1 소프트웨어 단위 시험

1) 소프트웨어 단위가 안전과 관련되었다면 이 하위절의 요구사항을 준수해야 한다.

2) 소프트웨어 단위 시험을 계획하고 명세하며 실행하여야 한다.

3) 소프트웨어 단위가 다음 사항을 구현한다는 것을 증명하기 위해 아래 표에 나열된 소프트웨어 단위 시험을 적용하여야 한다.

a) 단위 설계 명세서와의 부합
b) 하드웨어와 소프트웨어간 인터페이스 명세서와의 부합
c) 명세된 기능성

d) 의도하지 않는 기능이 없음에 대한 확신

e) 강건성

f) 기능을 뒷받침하기 위한 충분한 자원

방법		ASIL			
		A	B	C	D
a	요구사항 기반 시험 [a]	++	++	++	++
b	인터페이스 시험	++	++	++	++
c	결함 주입 시험[b]	+	+	+	++
d	자원 사용 시험[c]	+	+	+	++
e	모델과 코드 간 비교 시험(back-to-back test), 해당되는 경우[d]	+	+	++	++

+ : 매우 권장, + : 권장, o : 권장사항 없음

4) 소프트웨어 단위 시험에 대한 적절한 시험 케이스의 명세서가 가능하도록 하기 위해 시험 케이스는 아래표에 나열된 방법을 이용하여 생성되어야 한다.

방법		ASIL			
		A	B	C	D
a	요구사항 분석	++	++	++	++
b	동치 클래스(equivalence class)의 생성 및 분석	+	++	++	++
c	경계 값(boundary value)의 분석	+	++	++	++
d	오류 추측	+	+	+	+

+ : 매우 권장, + : 권장, o : 권장사항 없음

5) 시험 케이스의 완전성을 평가하고 의도하지 않은 기능이 없다는 것을 증명하기 위해 아래 표에 나열된 지표에 따라서 소프트웨어 단위 수준의 요구사항의 커버리지가 결정되고 구조적 커버리지가 측정되어야 한다. 달성된 커버리지가 불충분하다고 여겨지면 추가 시

험 케이스가 명시되거나 근거가 제공되어야 한다.

방법		ASIL			
		A	B	C	D
a	구문 커버리지	++	++	+	+
b	분기 커버리지	+	++	++	++
c	MC/DC (수정된 조건/결정 커버리지)	+	+	+	++

+ : 매우 권장, + : 권장, o : 권장사항 없음

6) 소프트웨어 단위 시험을 위한 시험 환경은 실제 구현하기 위한 대상 환경에 가능한 한 가깝게 구성해야 한다. 만약 소프트웨어 단위 시험이 대상 환경에서 실시되지 않으면, 대상 환경에서 추가적인 시험을 명시하기 위해 이후의 시험 단계 동안 소스와 목적 코드의 차이, 시험 환경과 대상 환경의 차이가 분석되어야 한다.

참고 표준 (Reference)	• ISO 26262-6, 9절 소프트웨어 단위 시험 (Software unit testing)

Chapter 3. 애플리케이션 공통 모듈 설계하기

소프트웨어 통합 및 시험

Attribute	Description
목표 (Objectives)	• 소프트웨어 엘리먼트를 통합 • 소프트웨어 아키텍처 설계가 임베디드 소프트웨어에 의해 구현된다는 것을 증명
입력물 (Prerequisite)	• 하드웨어와 소프트웨어간 인터페이스 명세서 • 소프트웨어 아키텍처 설계 명세서 • 안전 계획 • 구현된 소프트웨어 단위 • 소프트웨어 검증 계획 • 소프트웨어 검증 명세서 • 소프트웨어 검증 보고서
활동 (Activities)	• 소프트웨어 아키텍처 설계에 대해 소프트웨어 엘리먼트들 사이의 특별한 통합 수준 및 인터페이스를 시험

산출물 (Work product)

산출물명	QM	A	B	C	D	R:작성	A:승인	S:지원	I:통보	제출형태
[54] Software verification plan	○	○	○	○	○	Supplier				Full document
[55] Software verification specification	○	○	○	○	○	OEM			Supplier	Full document
[61] Embedded software	○	○	○	○	○	Supplier			OEM	Full document
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document

활동 요구사항 (Requirement of Activities)

- 다음은 ISO 26262 표준에서 제시하고 있는 소프트웨어 통합 및 시험 방법이다.

5.6.1 소프트웨어 통합 및 시험

1) 소프트웨어 통합 계획은 임베디드 소프트웨어가 완전히 통합될 때까지 소프트웨어 컴포넌트에 개별 소프트웨어 단위를 계층적으로 통합하는 단계를 설명하여야 하며 다음 사항을 고려해야 한다.

a) 소프트웨어 통합과 관련된 기능적 종속성
b) 소프트웨어 통합과 하드웨어와 소프트웨어간 통합 사이의 종속성

2) 소프트웨어 통합 시험은 **4.5**절에 따라 계획되고 명세되고 수행되어야 한다.

3) 아래 표에 나열된 소프트웨어 통합 시험 방법은 소프트웨어 컴포넌트와 임베디드 소프트웨어 모두 다음 사항을 구현한다는 것을 증명하기 위해 적용되어야 한다.

a) 7절에 따르는 소프트웨어 아키텍처 설계와의 부합
b) 하드웨어와 소프트웨어간 인터페이스의 명세서와의 부합
c) 명세된 기능성
d) 강건성
e) 기능을 지원하기 위한 충분한 자원

방법		ASIL			
		A	B	C	D
1a	요구사항 기반 시험	++	++	++	++
1b	인터페이스 시험	++	++	++	++
1c	결함 주입 시험	+	+	++	++
1d	자원 사용 시험	+	+	+	++
1e	해당되는 경우 모델과 코드 간 비교 시험(back-to-back test)	+	+	++	++
++ : 매우 권장, + : 권장, o : 권장사항 없음					

4) 소프트웨어 통합 시험 방법에 적합한 시험 케이스의 명세서를 사용하려면 아래 표에서 나열된 방법을 이용하여 시험 케이스가 생성되어

야 한다.

방법		ASIL			
		A	B	C	D
1a	요구사항 분석	++	++	++	++
1b	동치 클래스(equivalence class)의 생성 및 분석	+	++	++	++
1c	경계 값(boundary value)의 분석	+	++	++	++
1d	오류 추측	+	+	+	+
++ : 매우 권장, + : 권장, o : 권장사항 없음					

5) 시험 케이스의 완전성을 평가하고 의도하지 않은 기능이 없다는 것을 증명하기 위해 시험 케이스 별 소프트웨어 아키텍처 수준의 요구사항 커버리지가 결정되어야 한다. 필요 시, 추가 시험 케이스가 명시되거나 근거가 제공되어야 한다.

6) 시험 케이스의 완전성을 평가하고 의도하지 않은 기능이 없다는 것을 증명하기 위해 아래 표에 나열된 지표에 따라 구조 커버리지가 측정되어야 한다. 구현된 구조 커버리지가 불충분하다고 여겨지면 추가 시험 케이스가 설명되거나 근거가 제공되어야 한다.

방법		ASIL			
		A	B	C	D
1a	함수(function) 커버리지	+	+	++	++
1b	호출(call) 커버리지	+	+	++	++
++ : 매우 권장, + : 권장, o : 권장사항 없음					

7) 생산을 위한 릴리즈의 일부에 포함될 임베디드 소프트웨어가 명시된 모든 기능들을 포함했는지, 그리고 만약 명시되지 않은 기능들(디버깅이나 측정(instrumentation)을 위해 사용되는 코드를 포함)이 소프트웨어 안전 요구사항과의 부합함을 침해하지 않는다면 그런 기능들만을 포함했는지를 검증해야 한다.

8) 소프트웨어 통합 시험을 위한 시험 환경은 가능한 대상 환경에 가깝게 해야 한다. 소프트웨어 통합 시험이 대상 환경에서 실시되지 않으면 이후의 시험 단계 동안 대상 환경에

서 추가 시험을 명세하기 위해서 소스 코드와 목적 코드의 차이, 시험 환경과 대상 환경의 차이가 분석되어야 한다.

참고 표준 (Reference)	• ISO 26262-6, 10절 소프트웨어 통합 및 시험 (software integration and testing)

소프트웨어 안전 요구사항의 검증

Attribute	**Description**
목표 (Objectives)	• 임베디드 소프트웨어가 소프트웨어 안전 요구사항을 충족한다는 것을 증명
입력물 (Prerequisite)	• 소프트웨어 아키텍처 설계 명세서 • 안전 계획 • 소프트웨어 안전 요구사항 명세서 • 소프트웨어 검증 계획 • 소프트웨어 검증 명세서 • 소프트웨어 검증 보고서 • 통합 시험 보고서
활동 (Activities)	• 임베디드 소프트웨어가 대상 환경에서 요구사항을 충족한다는 것을 증명

산출물 (Work product)										
산출물명	**QM**	**A**	**B**	**C**	**D**	**R:작성**	**A:승인**	**S:지원**	**I:통보**	**제출형태**
[54] Software verification plan	○	○	○	○	○	Supplier				Full document
[55] Software verification specification	○	○	○	○	○	OEM			Supplier	Full document
[56] Software verification report	○	○	○	○	○	Supplier	OEM			Full document
활동 요구사항 (Requirement of Activities)										

- 다음은 ISO 26262 표준에서 제시하고 있는 하드웨어 통합 및 시험 방법이다.

5.7.1 소프트웨어 안전 요구사항 검증

1) 임베디드 소프트웨어가 소프트웨어 안전 요구사항을 충족한다는 것을 증명하기 위해 아래 표에 나열된 시험 환경에서 시험이 실시 되어야 한다.

방법		ASIL			
		A	B	C	D
1a	루프 내의 하드웨어	+	+	++	++
1b	전자 제어 단위 네트워크 환경	++	++	++	++
1d	차량	++	++	++	++
++ : 매우 권장, + : 권장, o : 권장사항 없음					

2) 소프트웨어 안전 요구사항의 실시 시험은 대상 하드웨어 상에서 실행되어야 한다.

3) 소프트웨어 안전 요구사항의 검증 결과는 다음과 관련하여 평가되어야 한다.

a) 예상 결과와 부합

b) 소프트웨어 안전 요구사항의 커버리지

c) 합격이나 불합격 기준

참고 표준 (Reference)	• ISO 26262-6, 11절 소프트웨어 요구사항의 검증 (Verification of software requirements)

소프트웨어 도구 사용에 대한 신뢰 확보

Attribute	Description
목표 (Objectives)	• 소프트웨어 도구가 기능안전 활동이나 업무를 수행하는데 적합하다는 근거를 제시하기 위함
입력물 (Prerequisite)	• 안전 계획 • 소프트웨어 도구가 사용되는 안전 수명주기 단계와 관련된 정보
활동 (Activities)	• 소프트웨어 도구 사용의 신뢰 확보 계획 수립 • 소프트웨어 도구 평가 • 소프트웨어 도구 사용의 신뢰 확보

산출물 (Work product)										
산출물명	QM	A	B	C	D	R:작성	A:승인	S:지원	I:통보	제출형태

[64] Software tool criteria evaluation report		○	○	○	○	Supplier				필요시 on-site 확인
[65] Software tool qualification report		○	○	○	○	Supplier				필요시 on-site 확인

활동 요구사항 (Requirement of Activities)

5.8.1 소프트웨어 도구 사용의 신뢰 확보 계획 수립

1) 다음의 내용을 포함하여 소프트웨어 도구의 사용이 계획되어야 한다.

- 소프트웨어 도구의 식별 및 버전 번호
- 소프트웨어 도구의 설정
- 소프트웨어 도구의 유즈 케이스
- 소프트웨어 도구가 실행되는 환경
- 만약 소프트웨어 도구가 오작동 중이고 해당 오류출력을 발생시킨다면, 위반될 수 있는 아이템이나 엘리먼트에 할당된 모든 안전 요구사항의 최대 ASIL
- 필요 시, 결정된 신뢰수준에 기반하여 소프트웨어 도구를 검증하는 방법

5.8.2 소프트웨어 도구 평가

1) 소프트웨어 도구에 대한 적절한 평가와 사용을 보장하기 위하여, 다음 정보가 반드시 있어야 한다.

- 소프트웨어 도구의 특징, 기능, 기술적인 속성에 대한 설명
- 해당되는 경우 사용자 매뉴얼 및 기타 사용지침
- 동작에 요구되는 환경을 설명함
- 해당되는 경우, 이상 작동 조건에서 소프트웨어 도구의 예상되는 행위에 대한 설명
- 알려진 소프트웨어 도구의 오작동, 적합한 보호장치, 해당되는 경우, 방지 또는 기타해결 수단에 대한 설명
- 소프트웨어 도구의 요구되는 신뢰수준을 판단하는 동안, 확인된 소프트웨어 도구의 오작동과 그에 해당하는 오류가 있는 출력을 검출하기 위한 수단

2) 분석에 의한 소프트웨어 도구 평가

- **[도구 영향(TI) 등급 결정]** 개발중인 안전관련 아이템이나 엘리먼트에서, 특정 소프트웨어 도구가 오류를 일으키거나 오류 탐지에 실패할 오류의 가능성
  - 상기 언급된 확률이 없다는 논거가 있으면 TI1이 선택되어야 한다.
  - 그 외의 모든 경우에는 TI2가 선택되어야 한다.
- **[도구 오류 검출(TD) 등급 결정]** 소프트웨어 도구가 오작동하고 해당 오류출력 생성을 방

지하는 수단 또는 소프트웨어 도구가 오작동하고 오류출력을 검출하기 위하여 사용되는 수단의 신뢰정도

• 오작동과 해당 오류출력이 방지되거나 검출되는 신뢰수준이 높으면, TD1이 선택되어야 한다.

• 오작동과 해당 오류출력이 방지되거나 검출되는 신뢰수준이 중간이면, TD2가 선택되어야 한다.

• 그 외의 모든 경우에는 TD3가 선택되어야 한다.

- **[소프트웨어 신뢰 수준(TCL) 결정]** 결정된 TI와 TD 등급을 통해 TCL 등급을 결정한다.

도구의 신뢰수준(TCL)의 결정

		도구 오류 검출		
		TD1	**TD2**	**TD3**
도구 영향	TI1	TCL1	TCL1	TCL1
	TI2	TCL1	TCL2	TCL3

5.8.3 소프트웨어 도구 사용의 신뢰 확보

- TCL1으로 분류된 소프트웨어 도구에 대해서는 요구되는 신뢰 확보 활동이 없다.
- TCL3으로 분류된 소프트웨어 도구의 경우 다음의 방법을 따라 신뢰 확보를 수행한다.

방법		**ASIL**			
		A	**B**	**C**	**D**
1a	사용으로 인하여 증가된 신뢰	++	++	+	+
1b	도구 개발 프로세스의 평가	++	++	+	+
1c	소프트웨어 도구의 타당성 확인	+	+	++	++
1d	안전 표준에 따라 개발	+	+	++	++
++ : 매우 권장, + : 권장, o : 권장사항 없음					

- TCL2로 분류된 소프트웨어 도구의 경우 다음의 방법을 따라 신뢰 확보를 수행한다.

방법		**ASIL**			
		A	**B**	**C**	**D**
1a	사용으로 인하여 증가된 신뢰	++	++	++	+
1b	도구 개발 프로세스의 평가	++	++	++	+
1c	소프트웨어 도구의 타당성 확인	+	+	+	++
1d	안전 표준에 따라 개발	+	+	+	++
++ : 매우 권장, + : 권장, o : 권장사항 없음					

Chapter 4. UML 작성

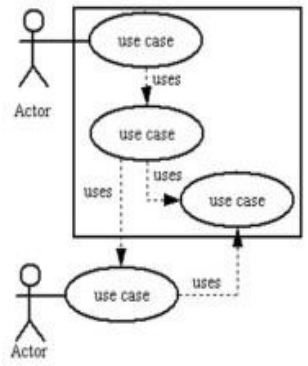
UML 은 'Unified Modeling Language'의 약자입니다. 무엇을 통합했기에 'Unified' 라는 말이 붙게 된 것일까요?
먼저, UML 의 역사를 간단히 살펴보면 아래 그림과 같습니다.



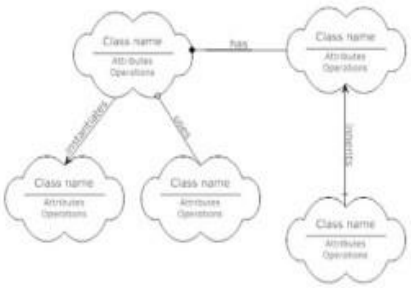
객체지향적 분석과 디자인에 대해 다양한 방면으로 실험적인 접근을 하던 방법론자들에 의해서 다양한 객체지향 모델링 방법이 등장하게 됩니다. 그 중에서도 Booch 의 Booch 방법론, Rumbaugh 의 OMT(Object Modeling Technique)와 Jacobson 의 OOSE(Object-Oriented Software Engineering) 방법론이 유명했습니다.
이렇게 많은 방법론이 존재하다보니, 서로 다른 방법론을 사용하는 조직 간에 모델 정보를 공유하기 위해선 각 조직에서 사용하는 방법론의 표현기호를 익히고 이해해야 했습니다. 상황이 이렇다 보니 모델을 표현하기 위한 동일한 표현 기호, 동일한 모델링 언어를 사용하는 것에 대한 필요성을 느끼게 됩니다.

UML 개발은 1994 년 후반에 Rational 사의 Grady Booch 와 Jim Rumbaught 에 의해 그들의 방법론인 Booch 와 OMT 의 통합으로 시작 되었습니다. 1995 년 가을 Ivar Jachobson 이 소속된 회사 Objectory 와 Rational 이 합병되면서 통합의 노력이 있었고 이에 Jacobson 의 OOSE 방법론이 통합 되었습니다.

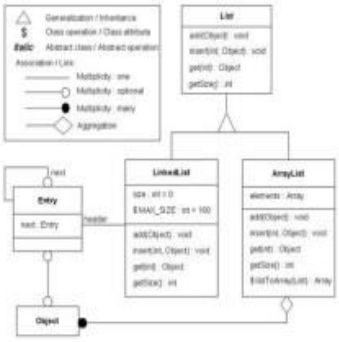
** 여기서 잠깐! Booch, Rumbaugh, Jacobson 의 모델링 언어는 어떤 모습이었는지 살펴볼까요?*



[Jacobson's Use Case diagram]



[Booch Class Diagram]



[Booch Object Diagram]

Booch, Rumbaugh, Jacobson 은 다음의 세 가지 이유로 통합된 모델링 언어를 만들게 되었습니다.

1. 각각의 방법론들이 이미 각자 독립적인 발전을 하고 있었다. 각 방법론에서 사용자에게 혼란을 일으킬 불필요한 부분을 제거하고 통합하여 같이 발전 시키는 것이 이치에 맞았다.

2. 완벽한 의미와 표기의 통일로써 안정된 객체지향 시장을 형성할 수 있다. 또한 프로젝트에 완벽한 모델링 언어를 제공하고 모델링 언어를 지원하는 툴들에게 더 나은 특징을 제공할 수 있게 된다.

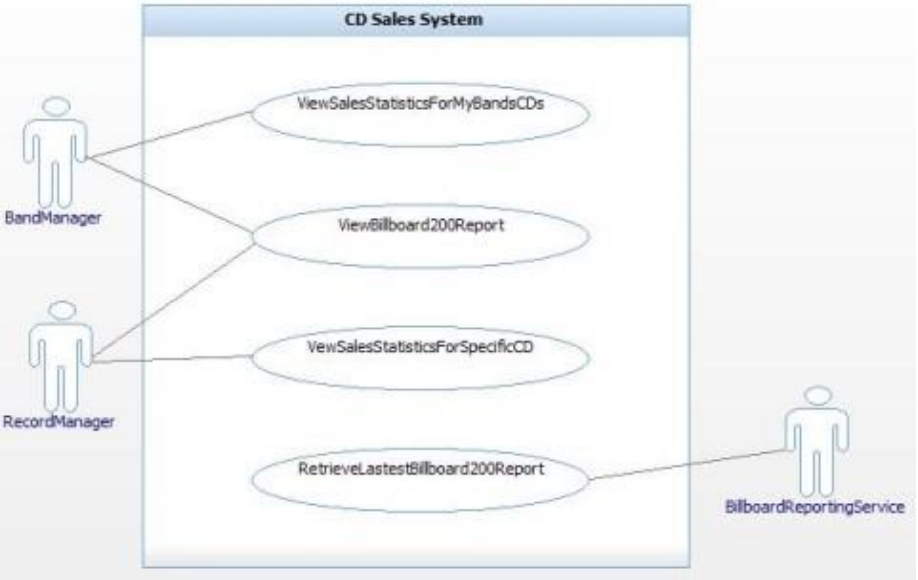
3. 각자의 방법론이 서로 보완하여 더 나은 발전을 이룰 수 있고 각자의 방법론이 해결할 수 없는 문제를 해결할 수 있도록 도와준다.

이들 세 사람은 모델링 언어를 통합하기 위해 다음의 목표를 두고 노력을 기울였습니다.
1. 객체지향 개념을 이용하여 소프트웨어 영역뿐만 아니라 시스템 영역도 모델링 할 수 있게 한다.
2. 실행 가능하거나 개념적인 산출물들을 명확하게 결합할 수 있게 한다.
3. 사람과 기계에 모두 유용한 모델링 언어를 만든다.

세 사람의 노력으로 1996 년 6 월에 UML0.9 문서가 공개되었으며, UML 협의회를 설립하여 강력하고 완전한 UML 을 정의하기 위한 행보를 진행하게 됩니다. UML 1.0 을 정의하는 일에 기여한 협력 업체로는 HP, I-Logix, Intelicop, IBM, Oracle, Microsoft 등등이 있습니다.
UML1.0 문서는 1997 년 1 월 OMG 에 표준안으로 제출 되게 되고, UML1.1 로 개정하여 1997 년 9 월 OMG 에서 표준안으로 채택 되게 됩니다.

Use Case Diagram

Use Case 는 시스템에서 제공한 기능 단위를 설명합니다. Use Case Diagram 의 주요 목적은 다른 Use Case 간 관계뿐만 아니라 주요 프로세스에 대한 "액터(actors)" (시스템과 인터렉팅 하는 사람)들과의 관계를 포함하여 개발 팀들이 시스템의 기능적 요구 사항들을 시각화 하는 데 있습니다.
Use Case Diagram 은 Use Case 그룹들을 보여줍니다. 완전한 시스템에 대한 모든 Use Case 이거나 관련된 기능을 가진 특정 Use Case 그룹(예를 들어, 보안 관리에 관련된 Use Case 그룹)일 수도 있습니다.
Use Case 다이어그램에 대한 Use Case 를 보여주려면 Diagram 중간에 타원을 그려서 타원의 중앙 또는 아래에 Use Case 이름을 부여합니다. Use Case Diagram 에 액터(시스템 사용자)를 그리려면 다이어그램의 왼쪽이나 오른쪽에 사람 모양을 그려 넣습니다.
Actor 와 Use Case 간 관계는 [그림 1]에 나타나 있습니다.



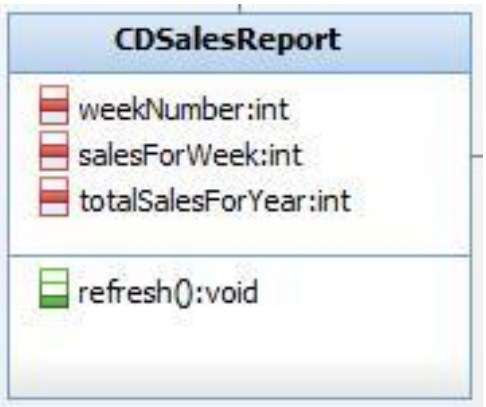
[그림 1] Use Case Diagram

Use Case Diagram 은 시스템의 고급 기능과 시스템의 범위를 설명하는데 사용됩니다.
[그림 1]의 Use Case Diagram 을 통해 시스템이 제공하는 기능을 쉽게 표현할 수 있습니다.
이러한 시스템에서는 밴드 매니저가 밴드가 발매한 CD 에 대한 판매 통계 리포트와 Billboard 200 보고서를 볼 수 있습니다.
또한 레코드 매니저는 특정 CD 에 대한 판매 통계 보고서와 Billboard 200 보고서를 볼 수 있습니다.
이 다이어그램에서는 Billboard Reporting Service 라고 하는 외부 시스템에서 우리 시스템이 Billboard 리포트를 전달하고 있다는 것도 볼 수 있습니다.
또한, 이 다이어그램에 Use Case 가 없다는 것은 시스템이 수행하지 않은 일을 보여주고 있는 것입니다.
예를 들어, 이 다이어그램은 밴드 매니저가 Billboard 200 의 다른 앨범들에 수록된 노래를 듣는 방식은 나와있지 않습니다. Billboard 200 에서 Listen to Songs 라는 Use Case 에 대한 어떤 레퍼런스도 볼 수 없습니다.
이것은 중요한 포인트 입니다.
이와 같은 다이어그램에 제공된 명확하고 간단한 Use Case 설명을 통해 프로젝트 관리자는 시스템에 필요한 기능이 존재하는지 여부를 쉽게 볼 수 있는 것입니다.

Class Diagram

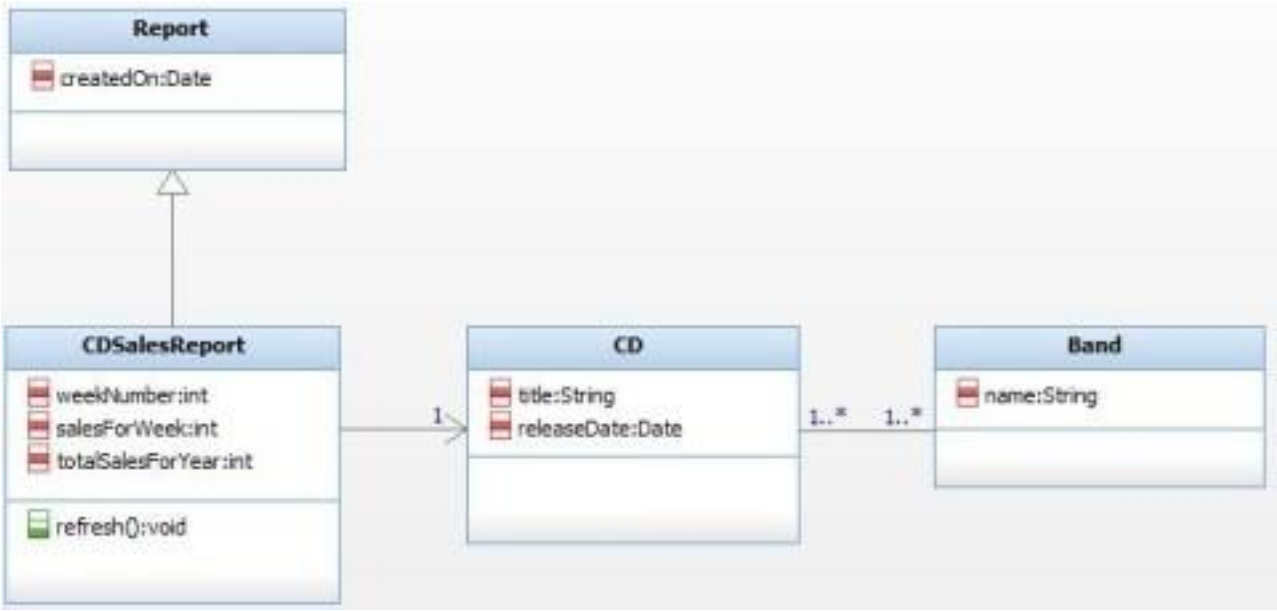
클래스 다이어그램은 다른 엔터티들(사람, 제품, 데이터)이 서로 어떻게 관계를 맺고 있는지를 표현 합니다.
다시 말해서 이것은 시스템의 정적 구조라고 할 수 있습니다. 클래스 다이어그램은 록밴드, 씨디, 라디오 연주를 논리적 클래스로 나타내는데 사용될 수 있습니다. 또는 대출, 주택 저당 대출, 자동차 대출, 이자율을 나타내는데도 쓰일 수 있습니다.
클래스 다이어그램은 주로 프로그래머들이 다루는 구현 클래스들을 보여주는데 쓰입니다. 구현 클래스 다이어그램은 논리적 클래스 다이어그램과 같은 클래스를 보여 줍니다. 하지만 이 구현 클래스 다이어그램은 같은 Attribute 로는 그릴 수 없습니다. Vectors 와 HashMaps 같은 것에 대한 레퍼런스를 갖고 있기 때문 입니다.

[그림 1]에서는 세 개의 섹션으로 클래스 다이어그램을 설명하고 있습니다. 위 섹션은 클래스의 이름을, 중간 섹션은 클래스의 Attribute 를, 가장 아래 섹션은 클래스의 Operation 을 보여주고 있습니다.



[그림 1] Class Diagram 의 Class 객체

거의 모든 개발자들은 이 다이어그램이 무엇을 하고 있는지를 알고 있습니다. 하지만 대부분의 프로그래머들은 관계도를 잘못 그리고 있습니다.
[그림 2]와 같은 클래스 다이어그램의 경우 상속 관계를 그릴 때에는 화살표 방향을 위로 향하게 하여 수퍼 클래스를 지시하게 하면서 화살표 모양은 완벽한 삼각형이 되도록 해야 합니다. 상관 관계는 두 클래스들이 서로를 인식하고 있다면 일직선이 되어야 하고, 클래스의 한 편만 알고 있는 관계라면 화살표 표시가 되어있는 선을 그어야 합니다.



[그림 2] 그림 1 의 Class 객체가 포함된 Class Diagram

[그림 2]에서 상속 관계와 두 개의 상관 관계를 보았습니다.
CDSalesReport 클래스는 Report 클래스에서 상속을 받았고, CDSalesReport 는 한 개의 CD 와 관련이 되어 있지만 CD 클래스는 CDSalesReport 에 대해 아무것도 모르고 있습니다. CD 와 Band 클래스는 서로에 대해 알고 있고 두 클래스는 서로 연관되어 있습니다.

Sequence 다이어그램

시퀀스 다이어그램은 특정 Use Case 에 대한 상세한 흐름이나 심지어는 특정 Use Case 의 일부분 까지도 보여줍니다. 시퀀스에서 다른 객체들 간의 호출관계를 보여주고 있고, 다른 객체들로의 다른 호출까지 상세하게 보여줄 수 있습니다.
시퀀스 다이어그램은 2 차원으로 표현합니다. 수직은 발생 시간 순서로 메시지/호출 시퀀스를 보여주고 수평은 메시지가 전송되는 객체 인스턴스를 나타내고 있습니다.
시퀀스 다이어그램은 그리기가 매우 간단 합니다. 다이어그램의 상단에 각 클래스의 인스턴스를 박스 안에 놓아 클래스 인스턴스(객체)를 구분 합니다. 박스 안에 클래스 인스턴스 이름과 클래스 이름을 스페이스/콜론/스페이스 " : "로 분리시킵니다. (예, myReportGenerator : ReportGenerator)

클래스 인스턴스가 메시지를 또 다른 클래스 인스턴스로 보내면 클래스 인스턴스를 받는 곳을 가리키는 화살표를 긋습니다. 그리고 그 라인 위에 메시지/메소드 이름을 적습니다.
중요한 메시지의 경우는 원래의 클래스 인스턴스를 다시 향하도록 점선 화살표를 그릴 수 있습니다. 점선 위에 리턴 값을 표기 합니다. 리턴 값을 포함하면 상세한 부분을 읽기 쉬운 장점이 있습니다.
시퀀스 다이어그램을 읽는 것은 매우 간단 합니다. 시퀀스를 시작하는 "드라이버(driver)" 클래스 인스턴스가 있는 왼쪽 상단 코너부터 시작 합니다. 그런 다음, 다이어그램 아래쪽을 각 메시지를 따라갑니다. [그림 1]의 시퀀스 다이어그램 예제에서 전송 메시지에 대한 리턴 메시지는 선택적인 것임을 기억해 주세요.



[그림 1] Sequence Diagram

[그림 1]의 시퀀스 다이어그램을 보면 CD Sales Report 가 어떻게 만들어지는지를 알 수 있습니다. ENV 객체가 우리의 드라이버 예제 입니다. ENV 는 메시지를 gen 이라고 하는 ReportGenerator 클래스 인스턴스로 보냅니다. 이 메시지는 generateCDSalesReport 라는 이름이 붙습니다. ReportGenerator 객체가 이 메시지 핸들러를 구현한다는 의미 입니다. 자세히 보면 generateCDSalesReport 메시지 이름은 괄호 안에 cdId 라는 파라미터가 있습니다.
gen 인스턴스가 generateCDSalesReport 메시지를 받으면 CDSalesReport 로 연속 호출을 하고 aCDReport 라고 하는 CDSalesReport 의 실제 인스턴스가 리턴 됩니다.
gen 인스턴스는 리턴된 aCDReport 인스턴스를 호출하면서 여기에 각 메시지 호출에 대한 매개변수를 전달 합니다. 시퀀스의 끝에서 gen 인스턴스는 Caller 였던 ENV 에 aCDReport 를 리턴합니다.

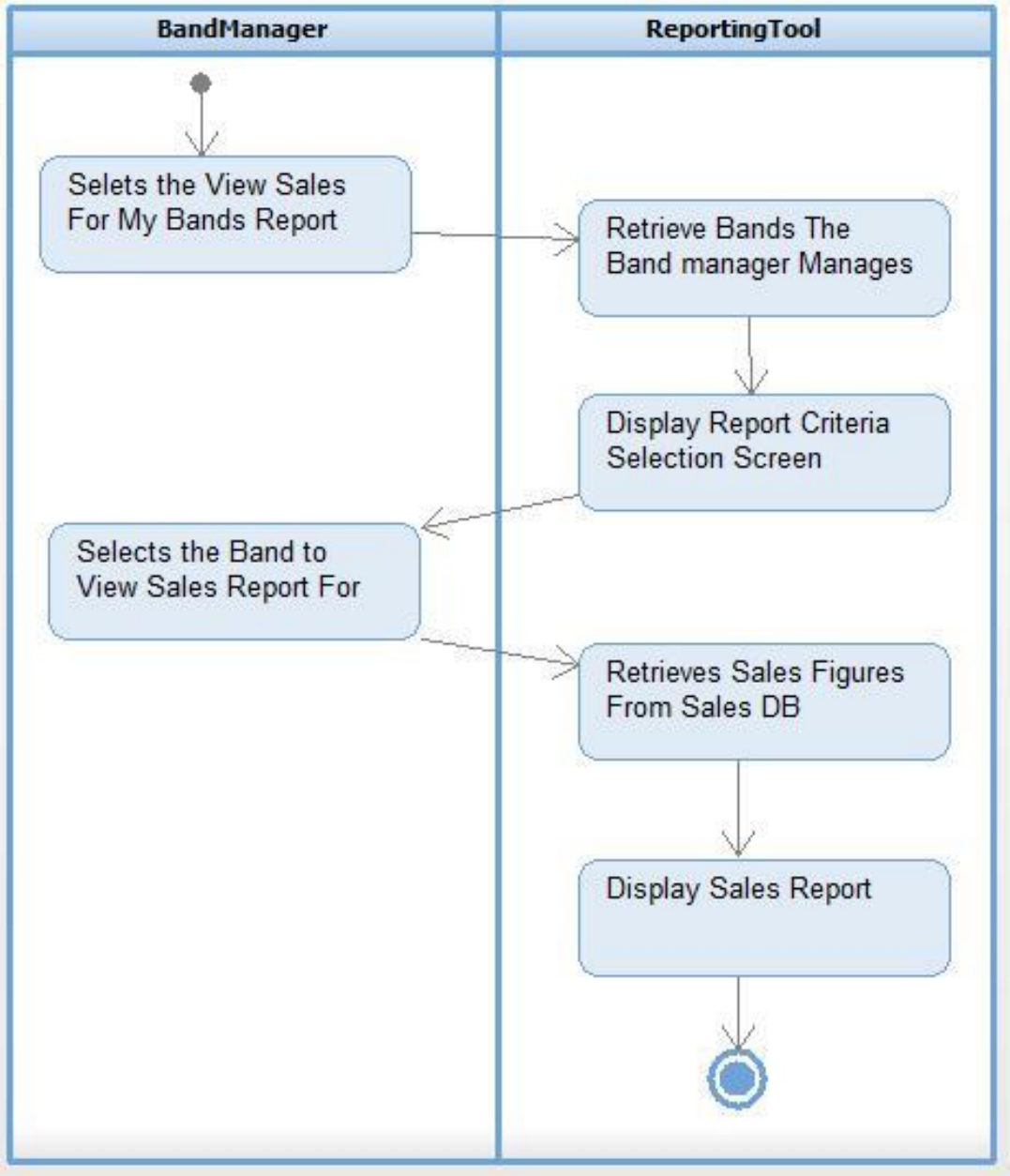
[그림 1]의 시퀀스 다이어그램은 전형적인 시퀀스 다이어그램을 상세히 설명한 것입니다. 하지만 충분히 이해할 수 있을 것입니다. 또한 초보 개발자들에게는 각 레벨 마다 시퀀스를 끊어서 이해하는 것도 좋은 방법입니다.

Activity Diagram

Activity Diagram 은 Activity 를 처리하는 동안 두 개 이상의 클래스 객체들 간 제어 흐름을 보여줍니다.
Activity 다이어그램은 비즈니스 단위 레벨에서 상위 레벨의 비즈니스 프로세스를 모델링 하거나 저 수준 내부 클래스 액션을 모델링 하는데 사용 됩니다. Activity 다이어그램은 기업이 현재 어떻게 비즈니스를 수행하는지, 또는 어떤 것이 비즈니스에 어떤 작용을 하는지 등의 고차원 프로세스를 모델링 할 때 적합 합니다. Activity 다이어그램은 언뜻 보기에 시퀀스 다이어그램 보다는 덜 기술적이기 때문에 비즈니스 마인드를 가진 사람들이 빠르게 이해할 수 있는 장점이 있습니다.

Activity 다이어그램의 표기법은 초기 Activity 에 연결된 실선으로 된 원에서 시작합니다. 이 Activity 는 모서리가 둥근 직사각형을 그려 그 안에 Activity 이름을 적어 넣으면서 모델링 됩니다. Activity 들은 이동 라인을 통해 다른 Activity 들에 연결되거나 결정 포인트의 조건에 제약을 받는 다른 Activity 들에 연결하는 결정 포인트로 연결될 수 있습니다. 모델링 된 프로세스를 종료하는 Activity 는 종료 포인트에 연결 됩니다. 이 Activity 들은 Swimlane 으로 그룹핑 될 수 있습니다. 이것은 실제로 Activity 를 수행하는 객체를 나타내는데 사용됩니다.

[그림 1]



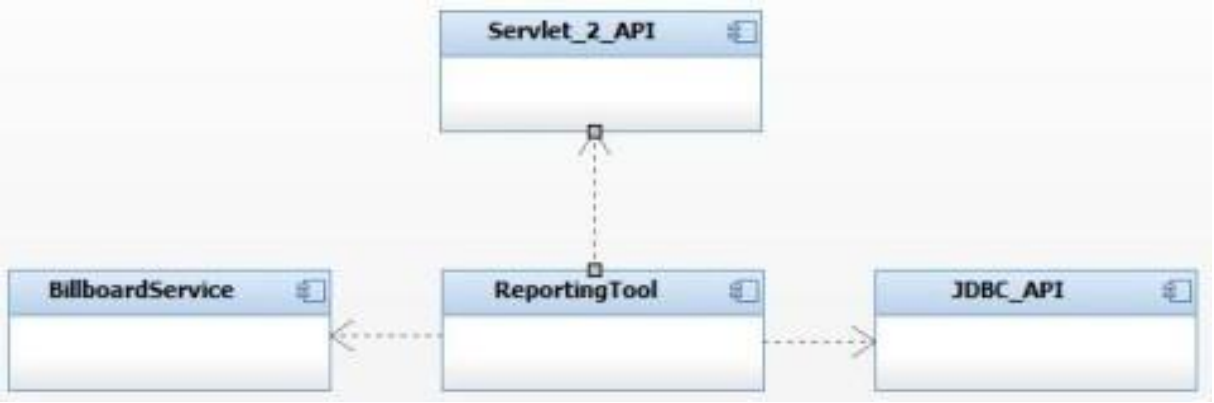
[그림 1] Activity Diagram

Activity Diagram 예제는 두 개의 객체(Band Manager, Reporting Tool)에 의한 Activity 제어를 나타내는 두 개의 SwimLane 으로 되어 있습니다. 프로세스는 한 밴드에 대한 판매 리포트를 보는 Band Manager 로 시작 합니다. Reporting Tool 은 사람이 관리하는 모든 밴드들을 검색하여 디스플레이하고 이중 한 개를 고를 것을 요청 합니다. Band Manager 가 한 밴드를 선택하면 Reporting Tool 은 판매 정보를 검색하여 판매 리포트를 디스플레이 합니다.

Component Diagram

Component Diagram 은 시스템을 물리적으로 볼 수 있도록 합니다. 이것의 목적은 소프트웨어가 시스템의 다른 소프트웨어 Component 들(예를 들어, 소프트웨어 라이브러리)에 대해 소프트웨어가 갖고 있는 종속 관계를 보여주는 것 입니다.
이 Diagram 은 매우 고급 레벨에서 볼 수 있거나 Component 패키지 레벨에서 볼 수 있습니다. (Component 패키지 레벨은 프로그래밍 언어에 중립적인 방식으로 .NET 의 네임스페이스(System.Web.UI)나 자바의 패키지(java.util) 같은 클래스 컨테이너 레벨을 참조하는 것이다)

Component Diagram 의 모델링은 이 예제에 잘 설명되어 있습니다.
[그림 1]은 네 개의 Component 인 Reporting Tool, Billboard Service, Servlet 2 API, JDBC API 를 보여주고 있습니다. Reporting Tool 에서 출발하여 Billboard Service, Servlet 2 API, JDBC API 로 가는 화살표는 Reporting Tool 이 이들 세 개의 Component 에 종속되어 있음을 나타냅니다.



[그림 1] Component Diagram

모델링 설계 효과와 한계점

과거에는 시스템 또는 소프트웨어 개발 단계는 요구사항으로부터 직접 구현이 이루어지는 단순한 과정을 거쳤습니다. 하지만 최근 요구사항과 구현 사이에 이루어지는 '설계' 활동의 중요성이 이미 강조되어 왔고, 제품 개발 생명주기에서 필수적인 단계가 되었습니다. 이미 많은 업체에서 설계를 위해 모델링 방식을 사용하고 있으며, SysML/UML 과 같은 모델링 표준들도 등장하게 되었습니다. 모델링 설계의 효과는 다음과 같이 정리할 수 있습니다.

1. 어플리케이션과 시스템의 복잡성을 단순화 시킨다.
2. 개발 초기에 위험요소 및 결함을 식별하여 유지보수 단계에서의 수정 비용을 절감할 수 있다.
3. 이해관계자들 간의 설계서 교환과 의사소통이 가능해진다.

모델링을 수행하는 업체 대다수는 편리한 설계를 위해 Rhapsody, Simulink 와 같은 모델링 도구들을 사용하고 있습니다. 하지만 많은 설계 도구들은 개발자 개별의 관점에 대부분의 기능들이 맞추어져 있어 팀 관점의 통합이 어렵다는 한계가 존재합니다.

1. 이해관계자 간의 설계 결과물 공유와 피드백이 어렵다.
2. 개발 프로세스가 진행되면서 각 단계로의 설계 링크 작업과 추적성 확보가 어렵고, 변경에 의한 영향도 분석이 어렵다.
3. 서로 다른 도구로 개발 된 설계 모델 간의 링크가 어렵다.
4. 다수의 설계 모델에 대한 보고서 생성에 많은 시간이 소요된다.
5. 대부분의 개발 부서는 격리된 환경에서 작업하기 때문에, 다른 부서와 설계 자산을 공유하는데 제한사항이 많다.

이와 같이 모델링 설계와 도구를 활용하는데 있어서 몇 가지 한계점이 존재하며, 이를 보완하기 위해 별도의 관리 방안과 솔루션이 필수적으로 요구됩니다.

설계 검토(Review)의 중요성

소프트웨어 공학에서 제시하는 주요한 시험(Testing) 활동으로는 인스펙션(Inspection), 정적 시험, 동적 시험 등이 있습니다. 이 중에서 인스펙션(Inspection) 기법은 짧은 시간에 가장 많은 결함을 찾아 낼 수 있고, 수정 비용도 가장 저렴한 방법으로 알려져 있습니다. AT&T, IBM 등을 포함한 실제 유수의 선진기업 사례에 따르면, 인스펙션 기법을 사용해 소프트웨어 오류를 최대 80%까지 판별할 수 있었으며, 생산량 또한 25~30% 향상시킬 수 있었다고 합니다. *(출처: "An encompassing life cycle centric survey of software inspection" – Oliver Laitenberger, Jean-Marc DeBaud)*

인스펙션 기법은 개발 부서에서 검토를 위한 그룹을 구성하고, 설계 구현한 결과물에 대해 정기적으로 검토를 실시하는 활동을 말합니다. 검토 시점은 개발 직후 또는 도중에 이루어지는 것이므로 결함 발견 시점이 상당히 빠르며, 수정도 바로 적용되어 비용 손실이 적을 수 밖에 없습니다. 인스펙션의 과정은 크게 검토를 위한 계획 단계, 검토 대상에 대한 참여자들의 준비 단계, 그리고 검토 결과에 대한 해결 단계로 진행됩니다. 인스펙션 활동을 하는데 주의할 점은 인스펙션의 목적이 결함을 발견하여 소프트웨어의 품질을 높이는 데에만 있다는 것을 명심해야 하고, 다른 사람에 대한 비판이나 성과 측정의 지표로 삼아서는 곤란합니다.

해외 선진 기업에서는 이미 인스펙션을 적용한 성공 사례들이 많이 나오고 있습니다. 하지만 국내에서는 인스펙션을 적용하는 기업이 많지 않습니다. 해외에 비해 국내에서 인스펙션이 제대로 실시되지 않는 가장 큰 이유는 개발 문화의 차이에 있습니다. 국내 대부분의 개발 조직과 개발자들은 자신의 코드를 다른 사람과 그룹에 공개하는 것을 굉장히 꺼려하며, 오픈 프로젝트에 익숙한 해외 개발자들의 의식에 비해 상당히 폐쇄적인 경향을 보입니다. 인스펙션 적용을 위해서는 가장 먼저 설계 구현 결과물에 대한 개방적 인식을 가지는 문화적 변화가 필요합니다.

국내에서도 몇몇 기업에서 인스펙션 기법을 적용하고 있지만 성공사례를 찾아보기가 어렵습니다. 그 이유는 다음 두 가지 유형으로 압축됩니다.

1. 인스펙션 활동 입증을 위해 흉내만 낼 뿐, 실제 인스펙션을 하지 않는 경우
2. 인스펙션 시간을 정기적으로 실시하긴 하나, 준비되지 않은 상태에서 시작하는 경우

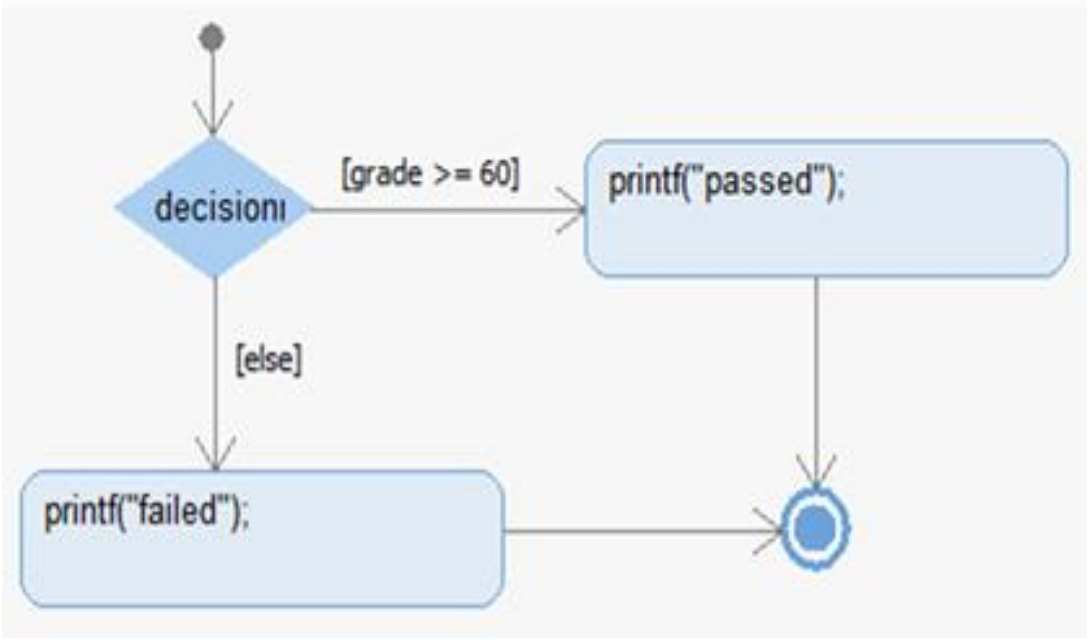
그러므로 인스펙션 활동은 구성원들간의 정확한 식별과 스케줄링 하에 이루어져야 하고, 사전 준비가 동반되어야 합니다. 개발자들에게는 이러한 별도의 활동과 준비시간이 부담스럽게 느껴질 수 밖에 없으므로, 인스펙션 활동을 지원하는 솔루션들이 존재하며 이에 대한 적극활용이 필요합니다.

Activity Diagram 에서의 제어 구조 표현 방법

UML Diagram 중에서 객체들간 제어 흐름을 표현할 때 주로 사용하는 Activity Diagram 에서 제어구조 표현을 하는 방법에 대한 몇 가지 샘플을 소개 합니다.

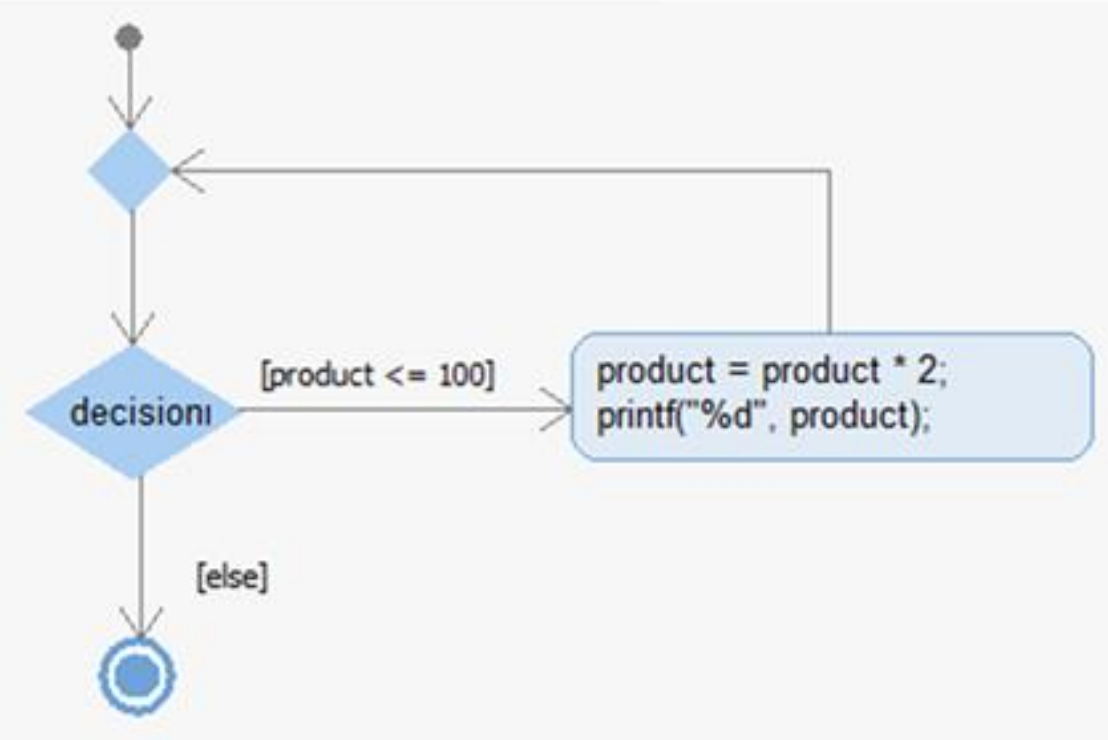
1. **if/else 선택 구조**

```
if (grade >= 60)
    printf("passed");
else
    printf("failed");
```



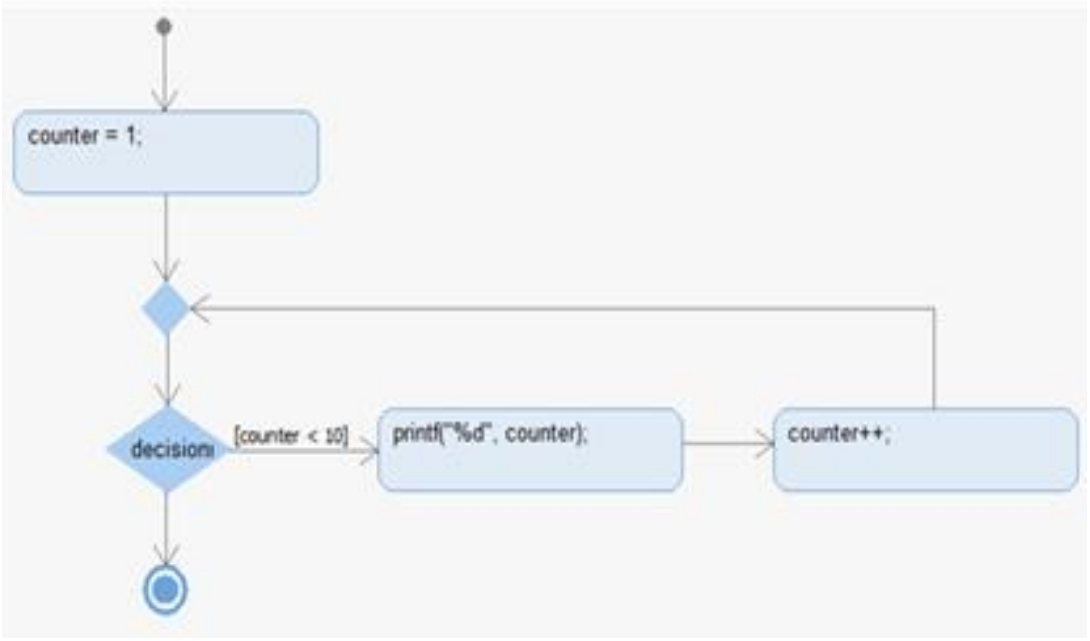
2. **while 반복 구조**

```
while(product <= 1000)
{
    product = 2 * product;
    printf("%d", product);
}
```



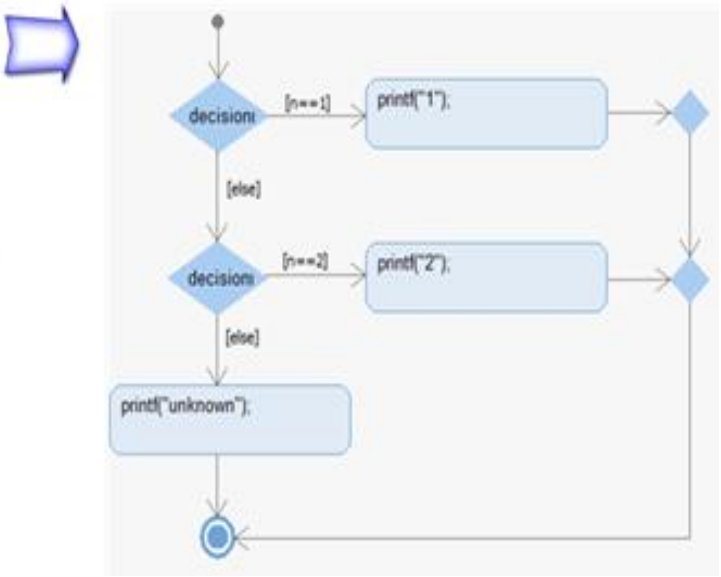
3. **for 반복 구조**

```
for(counter=1; counter<10; counter++)
{
    printf("%d", counter);
}
```



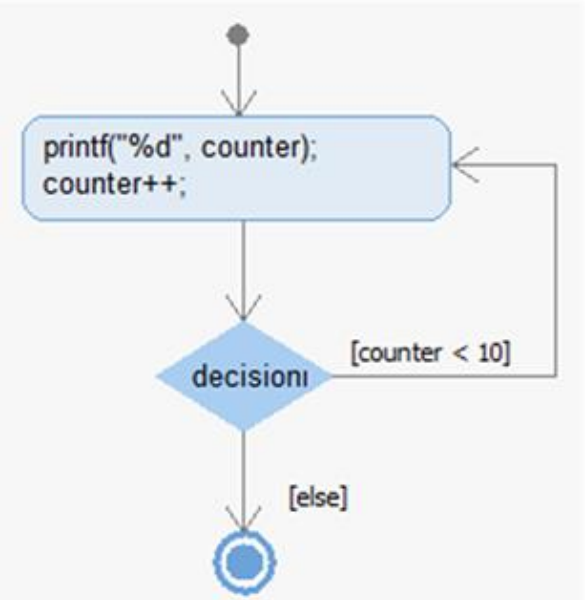
4. **switch 다중 선택 구조**

```
switch(n)
{
case 1:
    printf("1");
    break;
case 2:
    printf("2");
    break;
default:
    printf("unknown");
    break;
}
```



5. **do/while 반복 구조**

```
do {
    printf("%d", counter);
    counter++;
}while(counter < 10);
```



Part 2. 타 시스템 연동 설계하기

Chapter 5. Mybatis와 웹 애플리케이션 연동하기

1. MyBatis 설정 파일 (mybatis-config.xml) 생성하기

MyBatis의 설정파일은 getConnection( ) 메소드의 데이터베이스 연결정보를 대체하는 기능을 제공한다. 설정 파일에 대한 양식은 Mybatis 폴더에 동봉된 mybatis-3.4.5.pdf, 3page를 참조한다.

- Oracle Express Edition에서 SCOTT/TIGER 계정으로 연결을 설정한다 가정하자.

src/mybatis/mybatis-config.xml

```
01  <?xml version="1.0" encoding="UTF-8" ?>
02  <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
03  <configuration>
04    <environments default="development">
05      <environment id="development">
06  <!-- 트랜잭션 관리자를 설정, JDBC를 대체하는 의미에서 type을 JDBC로 지정한다. -->
07        <transactionManager type="JDBC" />
08
09        <!-- 데이터베이스 세부 설정 -->
10        <dataSource type="POOLED">
11          <property name="driver" value="oracle.jdbc.driver.OracleDriver" />
12          <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
13          <property name="username" value="scott" />
14          <property name="password" value="tiger" />
15        </dataSource>
16      </environment>
17    </environments>
18    <mappers>
19    <!-- SQL을 선언해 둔 XML이나 인터페이스 형태의 Mapper 위치를 지정한다 -->
20      <mapper resource="member/model/mapper/member-mapper.xml" />
21    </mappers>
22  </configuration>
```

1.1 MyBatis 객체 생성하기 - SqlSessionFactory

만들어진 설정파일(mybatis-config.xml)을 로드해서 MyBatis 객체를 생성한다

객체 생성 이후 MyBatis를 이용하여 데이터베이스 처리를 하려면 getSqlSessionFactory( ) 메소드를 이용하여 객체를 생성하고 API를 호출하여 사용하면 된다.

```
01  private SqlSessionFactory getSqlSessionFactory() {
02      String resource = "mybatis/mybatis-config.xml";
03      SqlSessionFactory factory = null;
04      try {
05          InputStream inputStream = Resources.getResourceAsStream(resource);
06          SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
07          factory = builder.build(inputStream);
08      } catch (IOException e) {
09      }
10      return factory;
11  }
```

1.2 SqlSession 객체 생성하기

SqlSessionFactory를 통해 SqlSession 객체를 생성할 때 여러 메소드가 오버로드 되어 있으나 기본적으로 2개의 메소드를 사용한다.

```
SqlSession openSession()
SqlSession openSession(boolean autoCommit)
```

Default 속성인 openSession()을 하게 되면, 자동 커밋 설정을 true 로 적용하기 때문에, false 속성을 주어 직접 관리하는 것을 권장한다.

5. MyBatis Mapper

1 MyBatis 요소

SqlSession 객체로 CRUD 구문을 실행하기 위한 메소드는 다음과 같다.

```
<T> T selectOne(String statement, Object parameter)
<E> List<E> selectList(String statement, Object parameter)
<K,V> Map<K,V> selectMap(String statement, Object parameter, String mapKey)
int insert(String statement, Object parameter)
int update(String statement, Object parameter)
int delete(String statement, Object parameter)
```

이 메소드들은 SQL 매핑 XML 파일에 정의된 SELECT, INSERT, UPDATE 그리고 DELETE 구문을 실행하기 위해 사용된다. 메소드 이름 자체가 그 역할을 설명하도록 명명되었으며, Select 구문의 경우 객체나 객체 리스트를, insert / update / delete의 경우 수행한 적용한 행의 개수를 리턴한다.

selectOne과 selectList의 차이점은 selectOne메소드는 오직 하나의 객체만을 리턴해야 한다는 것이다. 한개 이상을 리턴하거나 null 이 리턴된다면 예외가 발생할 것이다. 얼마나 많은 객체가 리턴될지 모른다면 selectList를 사용해야 하며, 객체의 존재여부를 체크하고 싶다면 개수를 리턴하는 방법을 사용하는 것이 효율적이다.

1.1 Mapper XML 파일 작성 – Select –

select구문은 마이바티스에서 가장 흔히 사용할 엘리먼트이다. 데이터베이스에서 데이터를 가져온다. 조회는 다음 예제처럼 단순한 경우에는 단순하게 설정된다.

```
<select id="selectMember" parameterType="string" resultType="hashmap">
  SELECT * FROM MEMBER WHERE ID = #{id}
</select>
```

여기서 구문의 이름은 selectMember이며, string타입의 파라미터를 받고, 결과를 hashMap에 저장한다. 파리미터에 대한 표기법은 '#{파라미터명칭}' 으로, 마이바티스에게 PreparedStatement를 만들도록 지시하는데, JDBC에서의 PreparedStatement 내 ' ? '와 그 쓰임이 동일하다.

속성	설명
resultType	반환되는 타입의 전체 클래스명이나 별명
resultMap	외부 resultMap 참조이름, 마이바티스의 가장 강력한 기능이다
useCache	구문의 결과에 캐시 사용 여부 지정, 디폴트는 true
fetchSize	지정된 결과의 수만큼 결과를 반환하는 값
resultSetType	FORWARD_ONLY, SCROLL_SENSITIVE, SCROLL_INSENSITIVE 중 선택

1.2 Mapper XML 파일 작성 – Insert, Update, Delete –

Insert, update, delete의 경우 사용 문법은 다음과 같다.
각각에 대한 파라미터로 원하는 객체를 지정하여 설정할 수도 있다.

```
<insert id="insertMember" parameterType="member.model.vo.Member">
  insert into MEMBER (id,username,password,email)
  values (#{id},#{username},#{password},#{email})
</insert>

<update id="updateMember" parameterType="member.model.vo.Member">
  update MEMBER set username = #{username},
     password = #{password},
     email = #{email}
  where id = #{id}
</update>

<delete id="deleteMember" parameterType="string">
  delete from MEMBER where id = #{id}
</delete>
```

속성	설명
parameterType	구문에 전달될 파라미터의 패키지 경로를 포함한 전체 클래스명이나 별칭
useGeneratedKeys	(insert, update) 데이터베이스에서 내부적으로 생성한 키 (MySQL, SQL Server과 같이 자동 증가 필드가 있는 경우만 해당), 디폴트는 false
keyProperty	getGeneratedKeys 메서드나 insert 문의 selectKey 하위 요소에 의해 반환된 키를 지정할 프로퍼티 지정
keyColumn	(insert, update) 생성키를 가진 테이블의 칼럼명을 셋팅. 키 칼럼이 테이블의 첫 번째 칼럼이 아닌 데이터베이스(PostgreSQL 처럼)에서만 필요

1.3 Mapper XML 사용 예제

실제 이를 사용하는 문법을 간략화 하여 설명해보면, 먼저 Member라는 사용자 정보를 조회할수 있는 'memberInfo' 쿼리가 작성된 mapper.xml 라는 파일이 있다고 하자. 그럼 이전에 선언한 getSqlSessionFactory()를

통해 SqlSession을 생성하여 다음과 같이 구문을 작성하여 사용한다.

src/member/model/mapper/member-mapper.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04 <mapper namespace="Member"> <!—namespace를 통해 내부 쿼리에 접근이 가능 -->
05
06  <!--  정의한 ID로 해당 쿼리의 정보를 가져온다. -->
07    <select id="memberInfo" parameterType="string" resultType="member.model.vo.Member">
08      SELECT * FROM MEMBER WHERE ID = #{userid}
09    </select>
10 </mapper>
```

member.model.dao.MemberDao.java

```
01 public Member selectMember(String userid) {
02       Member member = null;
03       SqlSession session = null;
04       try {
05          session = getSqlSessionFactory().openSession(false);
06
07          // 매퍼 파일 안에 작성된 쿼리문 실행시키고 결과 받는다.
08           member = session.selectOne("Member.memberInfo", userid);
09
10       } catch (Exception e) {
11          e.printStackTrace();
12       }finally{
13          session.close();
14       }
15       return member;
16    }
```

2 MyBatis 3.x를 이용한 CRUD 예제

mybatis-3.4.5.jar를 WEB-INF/lib 폴더에 넣는다
ojdbc6.jar 파일을 WEB-INF/lib 폴더에 넣는다

2.1 테스트를 위한 테이블 스키마

```
DROP TABLE MEMBER CASCADE CONSTRAINTS;

CREATE TABLE MEMBER(
    ID    VARCHAR2(15 BYTE),
    PASSWD    VARCHAR2(15 BYTE) NOT NULL,
    NAME    VARCHAR2(20 BYTE) NOT NULL,
    EMAIL    VARCHAR2(30 BYTE),
    GENDER    CHAR(1 BYTE) CHECK (GENDER IN ('M', 'F')),
    AGE    NUMBER,
    PHONE    CHAR(13 BYTE),
    ADDRESS    VARCHAR2(50 BYTE),
    ENROLL_DATE        DATE DEFAULT SYSDATE,
    CONSTRAINT PK_MEMBER PRIMARY KEY (ID)
);
```

2.2 MyBatis 설정파일 작성

src/mybatis/mybatis-config.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
      "http://mybatis.org/dtd/mybatis-3-config.dtd">
03  <configuration>
04    <settings>
05      <setting name="jdbcTypeForNull" value="NULL"/> <!-- Null 값 처리 -->
06    </settings>
07
08    <typeAliases> <!-- Class 별칭을 선언하여 모든 경로를 적어주지 않아도 된다. -->
09       <typeAlias type="member.model.vo.Member" alias="Member" />
10    </typeAliases>
11    <environments default="development">
12      <!-- DB 연동 설정 부, 'environment id'로 구분한다. -->
13      <environment id="development">
14        <transactionManager type="JDBC" />
15        <!—Mybatis는 transactionManager로 트랜잭션을 관리하므로
16              Autocommit 에 대한 설정을 할 수 있다. -->
17        <dataSource type="POOLED">
18          <property name="driver" value="oracle.jdbc.driver.OracleDriver" />
20          <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
```

```
21            <property name="username" value="student" />
22            <property name="password" value="student" />
23          </dataSource>
24        </environment>
25
26    <mappers>   <!—사용하고자 하는 mapper를 등록한다. -->
27        <mapper resource="member/model/mapper/member-mapper.xml" />
28    </mappers>
29  </configuration>
```

2.3 매퍼 파일 작성

src/member/mapper/member-mapper.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
03  <mapper namespace="Member">
04  <!-- Member클래스를 Alias로 지정하였다면, namespace에서 별칭으로 접근이 가능하다-->
05
06    <resultMap id="resultMember" type="Member">
07    <!-- property 는 필드명, column 은 DB 컬럼 명을 기술한다 -->
08      <id property="id" column="ID" />
09      <result property="passwd" column="PASSWD" />
10      <result property="name" column="NAME" />
11      <result property="email" column="EMAIL" />
12      <result property="gender" column="GENDER" />
13      <result property="age" column="AGE" />
14      <result property="phone" column="PHONE" />
15      <result property="address" column="ADDRESS" />
16      <result property="enrollDate" column="ENROLL_DATE" />
17    </resultMap>
18
19    <select id="memberInfo" parameterType="string" resultType="_int" resultMap="resultMember">
20       SELECT * FROM MEMBER WHERE ID = #{userid}
21    <!-- #{field}는 java.sql.PreparedStatement의 ?의 역할이다 -->
22    </select>
23
24    <select id="loginCheck" parameterType="Member"
```

```
25        resultType="Member" resultMap="resultMember">
26        SELECT * FROM MEMBER WHERE ID = #{id} AND PASSWD = #{passwd}
27      </select>
28
29  <insert id="insertMember" parameterType="Member" statementType="PREPARED">
30        INSERT INTO MEMBER VALUES(#{id}, #{passwd}, #{name}, #{email}, #{gender}, #{age}, #{phone}, #{address},
31  DEFAULT)
32      </insert>
33
34  <update id="updateMember" parameterType="Member" statementType="PREPARED">
35      UPDATE MEMBER SET PASSWD = #{passwd}, EMAIL = #{email}, AGE = #{age},
36        PHONE = #{phone}, ADDRESS = #{address} WHERE ID = #{id}
37    </update>
38
39  <delete id="deleteMember" parameterType="string" statementType="PREPARED">
40      DELETE FROM MEMBER WHERE ID = #{userid}
41      </delete>
42  </mapper>
```

2.4 Member VO 작성

member.model.vo.Member.java

```
01  package member.model.vo;
02
03  import java.sql.Date;
04
05  public class Member implements java.io.Serializable{
06      private static final long serialVersionUID = 1111L;
07      private String id;
08      private String passwd;
09      private String name;
10      private String email;
11      private String gender;
12      private int age;
13      private String phone;
14      private String address;
15      private Date enrollDate;
16
17      public Member(){}
18
19      public Member(String id, String passwd, String name, String email,
20          String gender, int age, String phone, String address) {
21        super();
22        this.id = id;
23        this.passwd = passwd;
24        this.name = name;
25        this.email = email;
26        this.gender = gender;
27        this.age = age;
28        this.phone = phone;
29        this.address = address;
30      }
31
32      public Member(String id, String passwd, String name, String email,
33          String gender, int age, String phone, String address, Date enroll_date) {
34        super();
35        this.id = id;
```

```
36            this.passwd = passwd;
37            this.name = name;
38            this.email = email;
39            this.gender = gender;
40            this.age = age;
41            this.phone = phone;
42            this.address = address;
43        }
44  /* ... Field 에 대한 Getter와 Setter 선언 ... */
45        @Override
46        public String toString(){
47            return this.id + ", " + this.passwd + ", " +
48            this.name + ", " + this.gender + ", "
49            + this.age + ", " + this.email + ", "
50            + this.phone + ", " + this.address
51            + ", " + this.enrollDate;
52        }
53  }
```

2.5 MemberDao 작성

member.model.dao.MemberDao.java

```
01  package member.model.dao;
02
03  import java.io.IOException;
04  import java.io.InputStream;
05
06  import org.apache.ibatis.io.Resources;
07  import org.apache.ibatis.session.SqlSession;
08  import org.apache.ibatis.session.SqlSessionFactory;
09  import org.apache.ibatis.session.SqlSessionFactoryBuilder;
10
11  import member.model.vo.Member;
12
13  public class MemberDao {
14      public MemberDao(){}
15        // Session 객체를 얻어 올 getSqlSessionFactory 메소드 구현
16      private SqlSessionFactory getSqlSessionFactory() {
```

```
17          String resource = "mybatis/mybatis-config.xml";
18          SqlSessionFactory factory = null;
19          try {
20              // InputStream으로 Mybatis의 설정 정보를 읽어 온다
21              InputStream inputStream = Resources.getResourceAsStream(resource);
22              // 읽어 온 Mybatis의 설정 정보를 바탕으로 SqlSessionFactoryBuilder를 생성한다
23              SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
24              factory = builder.build(inputStream);
25          } catch (IOException e) {
26              e.printStackTrace();
27          }
28          return factory;
29      }
30
31      public Member checkIdDup(String userid) {
32          Member member = null;
33          SqlSession session = null;
34          try {
35              // JDBC의 getSession()을 통한 Connection 객체 생성 처리와
36              // 동일한 의미를 가진 코드이다
37              session = getSqlSessionFactory().openSession(false);
38
39              // mapper 파일 안에 작성된 memberInfo 쿼리문을 실행하고 결과를 받는다
40              member = session.selectOne("Member.memberInfo", userid);
41          } catch (Exception e) {
42              e.printStackTrace();
43          }finally{
44              session.close();
45          }
46          return member;
47      }
48
49      public Member selectMember(Member m) {
50          Member member = null;
51          SqlSession session = null;
52          try {
53              session = getSqlSessionFactory().openSession(false);
54
```

```
55              member = session.selectOne("Member.loginCheck", m);
56              System.out.println(session);
57          } catch (Exception e) {
58              e.printStackTrace();
59          } finally{
60              session.close();
61          }
62          return member;
63      }
64
65      public int insertMember(Member m) {
66          int result = 0;
67          SqlSession session = null;
68          try {
69              session = getSqlSessionFactory().openSession(false);
70              result = session.insert("Member.insertMember", m);
71             // 세션 생성 시 AutoCommit을 false로 설정하였으므로
72             // commit / rollback을 직접 관리한다.
73              if(result > 0){
74                  session.commit();
75              } else {
76                  session.rollback();
77              }
78          } catch (Exception e) {
79              e.printStackTrace();
80          }finally{
81              session.close();
82          }
83          return result;
84      }
85
86      public int updateMember(Member m) {
87          int result = 0;
88          SqlSession session = null;
89          try {
90              session = getSqlSessionFactory().openSession(false);
91
92              result = session.update("Member.updateMember", m);
```

```
93              if(result > 0){
94                  session.commit();
95              } else {
96                  session.rollback();
97              }
98          } catch (Exception e) {
99              e.printStackTrace();
100         }finally{
101             session.close();
102         }
103         return result;
104     }
105
106     public int deleteMember(String userid) {
107         int result = 0;
108         SqlSession session = null;
109         try {
110             session = getSqlSessionFactory().openSession(false);
111
112             result = session.delete("Member.deleteMember", userid);
113             if(result > 0){
114                 session.commit();
115             } else {
116                 session.rollback();
117             }
118         } catch (Exception e) {
119             e.printStackTrace();
120         }finally{
121             session.close();
123         }
124         return result;
125     }
126 }
```

2.6 MemberService 작성

member.model.service.MemberService.java

```
01  package member.model.service;
02
03  import member.model.dao.MemberDao;
04  import member.model.vo.Member;
05
06  public class MemberService {
07      public MemberService(){}
08      // 기존 JDBC에서 쓰이던 Connection 연결을 Dao에서 관리하므로,
09      // 별도 Connection 정보를 담당하지 않는다.
10      public Member selectMember(Member m){
11          return new MemberDao().selectMember(m);
12      }
13
14      public int insertMember(Member m) {
15          return new MemberDao().insertMember(m);
16      }
17
18      public int updateMember(Member m) {
19          return new MemberDao().updateMember(m);
20      }
21
22      public int deleteMember(String userid) {
23          return new MemberDao().deleteMember(userid);
24      }
25
26      public Member checkIdDup(String userid){
27          return new MemberDao().checkIdDup(userid);
28      }
29  }
```

6. MyBatis 동적 SQL

1. 동적 SQL 작성하기

마이바티스의 가장 강력한 기능 중 하나는 동적 SQL을 처리하는 방법이다. 마이바티스에서는 if, choose (when, otherwise), trim (where, set), foreach를 동적 SQL 엘리먼트로 제공하는데, JSTL 과 사용 문법이 유사하다.

1.1 if

동적 SQL 에서 가장 공통적으로 사용되는 것으로 where의 일부로 포함될 수 있다.

```
<select id="findActiveBlogWithTitleLike" resultType="Blog">
  SELECT * FROM BLOG WHERE state = 'ACTIVE'
  <if test="title != null">
    AND title like #{title}
  </if>
</select>
```

1.2 choose, when, otherwise

자바의 switch, ORACLE의 CASE구문과 유사하며, 원하는 조건에 해당하는 경우를 조회할 수 있다.

```
<select id="findActiveBlogLike" resultType="Blog">
  SELECT * FROM BLOG WHERE state = 'ACTIVE'
  <choose>
    <when test="title != null">
      AND title like #{title}
    </when>
    <when test="author != null and author.name != null">
      AND author_name like #{author.name}
    </when>
    <otherwise>
      AND featured = 1
    </otherwise>
  </choose>
</select>
```

1.3 trim, where

본 예제를 통해 원하는 검색 결과를 얻는 상황을 생각해보자.

```
<select id="findActiveBlogLike"
      resultType="Blog">
  SELECT * FROM BLOG
  WHERE
  <if test="state != null">
    state = #{state}
  </if>
  <if test="title != null">
    AND title like #{title}
  </if>
```

```
    <if test="author != null and author.name != null">
      AND author_name like #{author.name}
    </if>
  </select>
```

만약 받은 파라미터가 어떤 조건에도 해당하지 않는다면, 다음과 같은 SQL이 생성될 것이다.

```
SELECT * FROM BLOG
WHERE
```

그럼 해당 SQL은 미완성되어 호출 시 에러가 발생 할 것이다.

만약 두번째 조건만 해당하는 경우에도 하기와 같은 SQL로 생성되어 실패할 것이다.

```
SELECT * FROM BLOG
WHERE
AND title like 'TEST'
```

이 같은 상황을 해결하기 위해 마이바티스에서는 다음과 같이 <where> 엘리먼트를 제공한다.

```
<select id="findActiveBlogLike"
     resultType="Blog">
  SELECT * FROM BLOG
  <where>
    <if test="state != null">
         state = #{state}
    </if>
    <if test="title != null">
        AND title like #{title}
    </if>
    <if test="author != null and author.name != null">
        AND author_name like #{author.name}
    </if>
  </where>
</select>
```

<where> 엘리먼트는 해당 SQL을 호출할 경우 WHERE 조건을 추가하며, 만약 AND 나 OR로 시작하는 조건으로 시작한다면 해당하는 접두사를 지워버린다. 만약 <where> 엘리먼트로 해결되지 않는다면, trim을 사용하여 구현할 수도 있다.

```
<trim prefix="WHERE" prefixOverrides="AND |OR ">
  ...
</trim>
```

1.4 foreach

동적 SQL 에서 반드시 필요한 것은 collection에 대한 반복처리일 것이다. 이러한 처리에 대해서 마이바티스는 Foreach를 통해 List나 Set, Map과 같은 Collection 파라미터를 받아 처리할 수 있다.

```
<select id="selectPostIn" resultType="domain.blog.Post">
    SELECT *
    FROM POST P
    WHERE ID IN
   <foreach item="item" index="index" collection="list" open="(" separator="," close=")">
          #{item}
   </foreach>
</select>
```

엘리먼트 내부에서 각각 item, index 변수를 통해 단일 객체 접근이 가능하며, Map을 사용할 경우, index는 key객체가 되고 항목은 value객체를 의미하게 된다. 또한 열고 닫는 문자열로 명시할 수 있고 반복간에 둘 수 있는 구분자도 추가할 수 있다.

Chapter 6. 오픈 API와 웹 애플리케이션 연동하기

6. 오픈 API 란 무엇인가?

시스템 분산과 통합

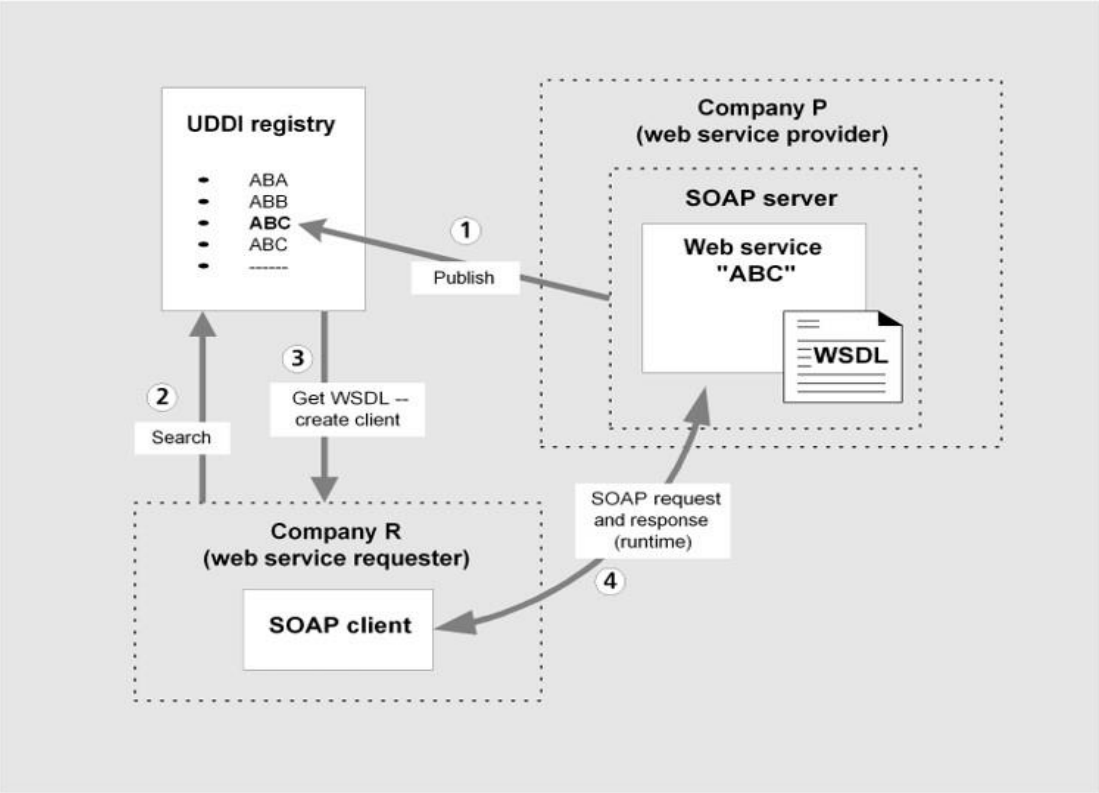
기업의 정보시스템은 매우 다양한 어플리케이션, 기술로 구성되어 있다. 메인프레임을 주로 사용하던 시기에는 대다수의 정보시스템이 메인프레임에 포함되어 있었다. 일부 단위 시스템은 업로드/다운로드 등으로 수작업 처리를 하거나, 핵심 시스템인 메인프레임에 영향을 최소화 하기 위하여 단방향 비동기 시스템 연동을 하고 있었다.

동기 처리는 동기화 시간 동안 시스템 자원이 강제로 할당되어 단위시스템의 장애 발생 시 메인프레임의 메인 처리에 영향을 줄 수 있다. 또한 기술과 비즈니스 환경이 빠르게 발전하고 정보화 사회가 가속화되면서 "Time to Market" 이 중요해지기 시작하였고 해당 업무에 전문화, 최적화 하기 위하여, 차세대 시스템 혹은 시스템 다운사이징 과정을 거쳐 고유의 IT 기술로 구현되어 수많은 단위 시스템으로 나뉘어 지게 되었다. 기업 내 수십 혹은 100 개 이상의 단위 시스템 간의 효율적인 연동이 이슈가 되었으며, MCI, EAI/ESB, FEP 등의 채널 및 어플리케이션 통합 기술이 발전하게 되었다.
이러한 연계 미들웨어의 요소 기술인 분산 시스템 연동 기술은 2000 년대 초반까지 CORBA, EJB, RMI, COM 등 원격 객체/컴포넌트 통신 방식이(Component Oriented 혹은 Object-Oriented) 활발히 연구되었으나, Stub/Skeleton 객체 등을 통하여 복잡하게 통신하여야 했으므로 점점 인기가 시들어져 가고, HTTP 기반의 웹서비스 기술을 활용하여 비즈니스적인 의미를 가지는 서비스 기능 단위로 묶어서 분산 시스템 연동을 하는 SOA(Service Oriented Architecture)가 유행하게 되었다.

SOAP API

SOA 웹서비스는 원격 서비스를 호출하는 개념이며, 대표되는 것이 SOAP(Simple Object Access Protocol) API 이다.

SOAP API 는 SOAP Envelope, SOAP Header, SOAP Body 로 구성된 XML 형식으로 구성되어 메시지의 사이즈가 커져서 시스템의 자원의 소모가 상대적으로 많으며, UDDI 에 WSDL 을 배포하고 검색할 수 있게 구성해야 하므로 아키텍처도 복잡하다.



[그림 1] SOAP API 퍼블리싱 및 호출 개념도

REST API

반면에, SOAP 보다 간결한 아키텍처 스타일로 최근 많이 사용되고 있는 REST(REpresentational State Transfer) API 는 원격 자원을 호출하는 개념인 ROA(Resource Oriented Architecture) 웹서비스이다.

REST 는 분산 하이퍼미디어 시스템(예: 웹)을 위한 아키텍처 스타일이고 REST API 는 REST 아키텍처 스타일을 따르는 API 이며, 일부 아키텍처 제약을 따르는 것을 RESTful API 라고 한다. REST 창시자인 Roy Fielding 은 "하이퍼텍스트를 포함한 self-descriptive 한 메시지의 uniform interface 를 통해 리소스에 접근하는 API" 라고 정의하였다.

Uniform Interface 의 제약조건 다음의 4 가지 이다.

- identification of resources
- manipulation of resources through representations
- self-descriptive messages
- hypermedia as the engine of application state (HATEOAS)

식별되어(Identification) 요청된 Resource 를 실행해 봤더니(manipulation), 그 시점에 해당 Resource 를 대표하는(Representation) 상태를(State) 응답 내용 자체만으로도 처리 결과가 이해가 가능한 메시지

형태를 만들고(self-descriptive messages), 응답 시점에 호출 가능한 어플리케이션을 Hyperlink 형태로 포함시켜(HATEOAS) 전송하는(Transfer) API 라는 것이다.

```
GET /accounts/12345/ HTTP/1.1
Host: bank.example.com
Accept: application/vnd.acme.account+json
...
```

```
HTTP/1.1 200 OK
Content-Type: application/vnd.acme.account+json
Content-Length: ...

{
    "account": {
        "account_number": 12345,
        "balance": {
            "currency": "usd",
            "value": 100.00
        },
        "links": {
            "deposit": "/accounts/12345/deposit",
            "withdraw": "/accounts/12345/withdraw",
            "transfer": "/accounts/12345/transfer",
            "close": "/accounts/12345/close"
        }
    }
}
```

[그림 2] REST API 요청 및 응답 예시

Account 라는 자원에 GET 이라는 조작을 했을 때 표현되는 상태를(balance, currency, value) 응답 메시지만으로도 이해 가능하게끔 쉽게 기술하고(200 OK, JSON), 또한 이 리소스의 상태를 바꿀 수 있는 하이퍼링크를(deposit, withdraw, transfer, close) 포함시켜 보낸 것이다.
SOAP 에서는 WSDL (XML)을 분석하여 이에 맞는 SOAP 메시지(XML)를 요청하고 응답을 받아 해석해야 하는 반면에, REST API 는 친숙한 HTTP URI 를 간결한 JSON 형태로 호출하고 응답을 받을 수 있어 아키텍처가 단순하고 사용하기 편하며 실행 속도가 빠르다.

API 의 역할 별 분류

API 를 어떤 목적으로 사용하는가에 따라 다음과 같이 나눌 수 있으며, 오픈 플랫폼에서는 대외적으로 Open API, Partner API, Composite API 를, 내부 시스템에서는 Internal API, Composite API 를 주로 활용한다.

Open API: 혹은 Public API, 외부에서 제약없이 호출할 수 있도록 공개된 API. 오픈 플랫폼에서 기본적으로 제공되어야 할 API 형태이다.
Partner API: 혹은 제휴 API, 핀테크기업 등 제휴업체와 사전 협의 후 제작하여 제공하며 이용 라이선스, 권한 등이 별도로 필요하다.

Internal API: 혹은 Private API, 기업 내부 시스템에서만 사용될 수 있으며, ESB 시스템이나 내부용 API Gateway 에 등록되어 호출되거나, API 서버에 직접 등록되어 Client 에서 호출될 수도 있다.
Composite API: 복수의 API 를 한번에 동기적으로 호출하는 방식으로 처리 속도를 높일 수 있으며, 개별 API 의 성공, 실패를 하나의 Response 에 Composite Subrequest 로 구별하여 되돌려준다.

```
{
"compositeRequest" : [{
  "method" : "POST",
  "url" : "/services/data/v38.0/sobjects/Account",
  "referenceId" : "refAccount",
  "body" : { "Name" : "Sample Account" }
  },{
  "method" : "POST",
  "url" : "/services/data/v38.0/sobjects/Contact",
  "referenceId" : "refContact",
  "body" : {
    "LastName" : "Sample Contact",
    "AccountId" : "@{refAccount.id}"
    }
  }]
}
```

[그림 3] Salesforce 의 Composite API 호출 예시

7. 오픈 API 디자인과 문서화

웹 API 디자인 원칙

1. 개발보다 협업 설계를 우선하라(Design-first Approach)

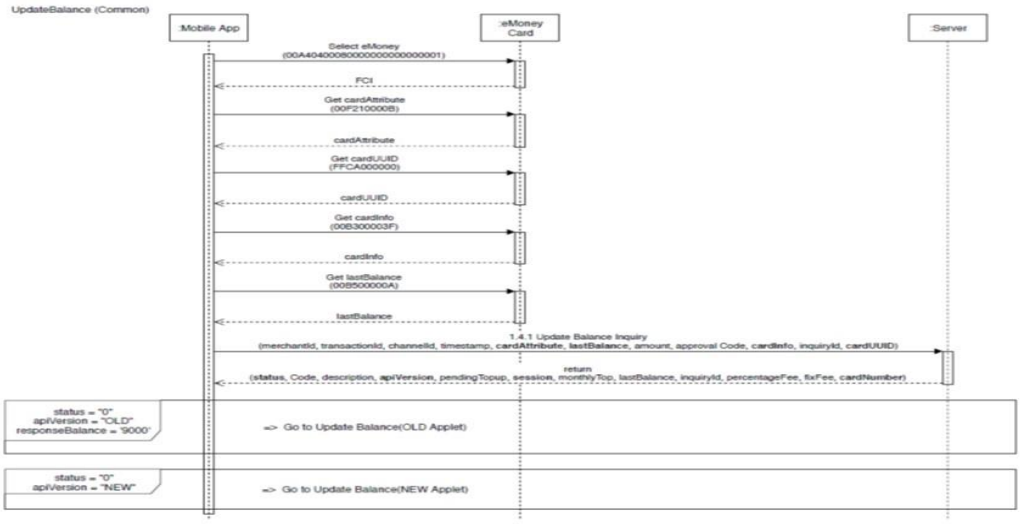
당연한 이야기 일 수 있지만 API 개발보다 설계부터 진행해야 한다. 일반적인 어플리케이션 개발 시에는 사용자로부터의 요구사항을 분석하는 것에 익숙하지만, API 는 개발자가 어플리케이션을 통하여 사용하는 시스템 자원이라는 선입견 때문에 서버 시스템이 제공할 수 있는 기능 위주로 API 를 먼저 개발하고 API 스펙을 만들어 내는 경향이 있다.

또한 Spring 프레임워크의 Annotation 을 통하여 swagger 형식의 API 문서를 자동 생성해낼 수 있기도 하다. 하지만 해당 API 를 누가 사용하는지 알 수 있는가? 내부 시스템 간 API 라도 다양한 어플리케이션에서 사용될 수 있기 때문에 API 동작을 확정하는게 쉽지 않다. 무엇보다도 API 는 협업의

도구이므로, 우선적으로 API 를 문서화 후 고객, 상품, 마케팅 등 다양한 조직과 API 협업 설계를 하는게 중요하다. 즉, 개발자가 아닌 현업, 유관기관, 시장조사 등 다양한 이해관계자들을 참여시켜야 API 활용성을 높일 수 있다.

2. API 들 간에 서비스 선후행 처리 흐름이 있다면 API Sequence Diagram 을 작성하여 제시하는 것이 필요하다

개별 API 사양에 기록된 전제조건, 처리 결과 상태 등의 항목 만으로는 API 간 처리 흐름이 파악하기 쉽지 않다. 플랫폼에서 이해관계자간 서비스를 융복합하여 제공하는 제휴 API 의 경우에는 서비스 흐름 및 처리 조건 등이 복잡해 질 수 있으므로 흐름을 쉽게 파악할 수 있는 Sequence Diagram 이 유용하다. 물론 계좌 잔액조회와 같이 자체로 처리가 완결되는 API 는 예외가 될 것 이다.



[그림 1] API Sequence diagram 예시

3. API 의 응답 헤더에 HTTP 상태코드 200 과 함께 에러메시지를 같이 보내지 말자

API 의 비즈니스 처리가 정상적으로 되지 않았다 해도 HTTP 호출은 정상적으로 이루어졌기 때문에 HTTP 상태코드를 200 으로 설정하고 API 응답의 에러메시지 항목에 에러 메시지를 설정하는 경우가 있다. 실제로 이런 사례를 경험하였다. 하지만 이럴 경우 API 사용자 입장에서는 정상처리 된 건지 아닌지 혼란스러울 수 있다. 이런 경우 HTTP 상태코드는 400 혹은 500 으로 설정하고 별도의 에러메시지를 설정하는 것을 권고한다.

HTTP 상태 코드(오류 유형)	오류 발생 원인	해결 방법
400(요청 변수 확인)	필수 요청 변수가 없거나 요청 변수 이름이 잘못된 경우	API 레퍼런스에서 필수 요청 변수를 확인합니다.
401(인증 실패)	클라이언트 아이디와 클라이언트 시크릿이 없거나 잘못된 값인 경우	내 어플리케이션 메뉴에서 클라이언트 아이디와 클라이언트 시크릿을 확인합니다.
403(호출 금지)	요청 변수의 값을 URL 인코딩으로 변환하지 않고 전송한 경우	API 레퍼런스에서 해당 요청 변수를 URL 인코딩으로 변환해야 하는지 확인합니다.
404(API 없음)	API 요청 URL이 잘못된 경우	API 요청 URL에 오류가 있는지 확인합니다.
405(메서드 허용 안 함)	HTTP 메서드를 잘못 호출한 경우. POST 방식으로 호출해야 하는데 GET 방식으로 호출한 경우 등입니다.	API 레퍼런스에서 HTTP 메서드를 확인합니다.
429 (호출 한도 초과 오류)	오픈 API를 호출할 때 하루 허용량을 초과한 경우	하루 허용량 초과 시 제휴 신청을 합니다.
500(서버 오류)	API 호출은 정상적으로 했지만, API 서버 유지 보수나 시스템 오류로 인한 오류가 발생한 경우	개발자 포럼에 오류를 신고합니다.

[그림 2] Naver 오픈 API 오류코드 일부 발췌

API 문서화(Specification)

API 설계 시 여전히 Excel 을 활용하는 곳도 있으나, Git 으로 관리할 수 없어 API 버전관리뿐만 아니라, 코드로부터 자동화 할 수 없고 즉시 실행해 볼 수도 없어 협업에 어려움이 있다.

이러한 것들을 가능하게 해주는 REST API 모델링 언어 및 도구는 다음과 같다.

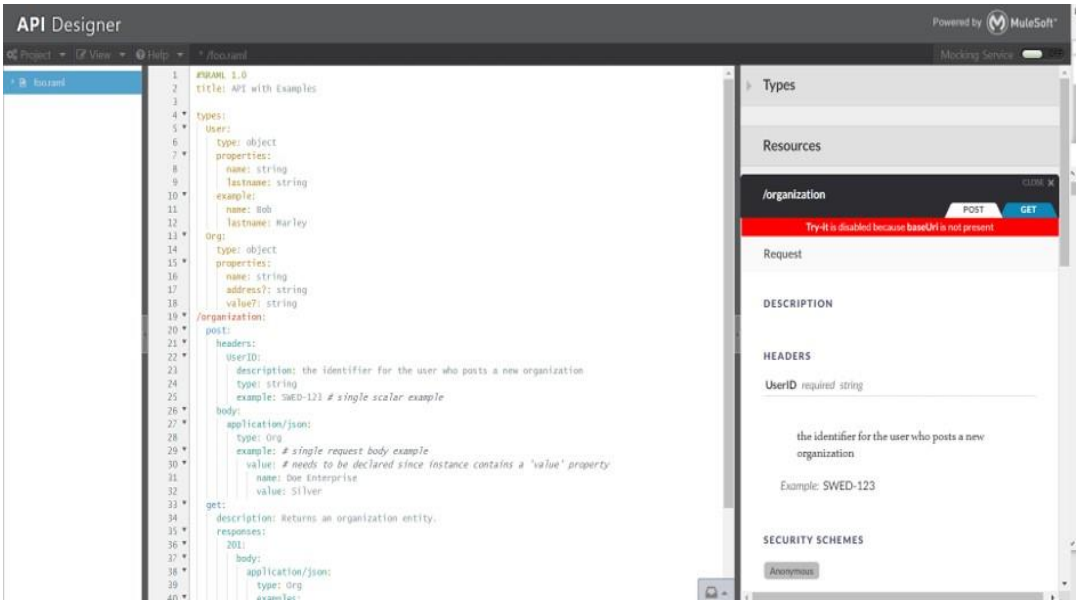
1. RAML(RESTful API Modeling Language)

RESTful API 를 설계하고 개발할 수 있는 YAML(YAML Ain't Markup Language) 기반의 모델링 언어이다. Apache 라이선스 2.0 에 따른다. RAML 을 지원하는 업체는 Mulesoft, AngularJS, Intuit, Box, PayPal, Programmable Web and API Web Science, Kin Lane, SOA Software, Cisco 이다.

2016 년 5 월 16 일 발표된 RAML 1.0 은 현재, YAML 1.2, JSON 포맷을 사용할 수 있으며, API-Designer, API Workbench 등의 Tool 로써 작성할 수 있다.

RAML 1.0 specification : https://github.com/raml-org/raml-spec/blob/master/versions/raml-10/raml-10.md/

다음은 node.js 만 설치되어 있으면 npm install 명령어로 설치하여 웹 기반으로 간단히 실행할 수 있는 API Designer 로 작성된 RAML API 예시이다.



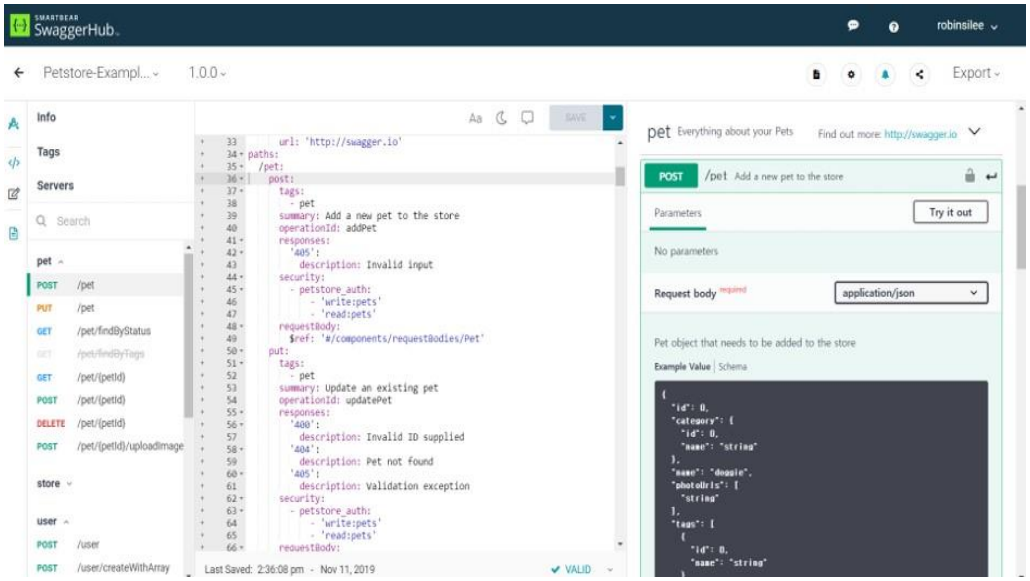
[그림 3] API designer 에서의 RAML 1.0 API 예시

2. OAS 3.0 (OpenAPI Specification 3.0)

2015 년에 Swagger Specification 에서 OpenAPI Initiative 로 기부(OAS 2.0) 되었으며, 2017 년 7 월 3.0 버전이 메이저 업그레이드 되었고 현 시점에서는 2018 년 10 월 8 일 발표된 3.0.2 버전이 최신이다. http://spec.openapis.org/oas/v3.0.2

Google, Microsoft, IBM, Paypal 의 지원을 받고 있고, RAML 의 강력한 후원자 였던 MuleSoft 가 리눅스 재단의 OAI 에도 2017 년 4 월 참여하였다.

YAML 과 JSON 포맷을 사용할 수 있으며, Swagger Editor 를(https://swagger.io/tools/swaggerhub/) 통해서 REST API 를 설계, 빌드, 자동문서화 및 실행해볼 수 있다.



[그림 4] Swagger Hub 에서의 OAS 3.0 API 예시

3. API Blueprint

MIT 라이선스의 오픈소스인 API Blueprint 는 현재 버전은 Format 1A revision 9 이며, JSON, YAML 뿐만 아니라, Markdown 문법도 지원하고 있다.

API Blueprint Specification :

https://github.com/apiaryio/api-blueprint/blob/master/API Blueprint Specification.md

참고

1. Markdown : 일반 텍스트 문서 양식이나 쉽게 HTML 또는 XHTML 로 변환이 가능하다.

https://simhyejin.github.io/2016/06/30/Markdown-syntax/

2. YAML : HTML 이 문서 표현에 집중한다면, YAML 은 데이터 자체를 중요시 한다는 의미이다.

[JSON]

```json
{
  "phone number": [
    {
      "office": "02-300-3000"
    },
    {
      "home": "02-200-2000"
    },
    {
      "mobile": "010-2000-2000"
    }
  ],
  "kids": [
    {
      "age": 15,
      "name": "kim younghee"
    },
    {
      "age": 12,
      "name": "kim youngsu"
    }
  ]
}
```

[YAML]

```yaml
phone number:
  - office: 02-300-3000
  - home: 02-200-2000
  - mobile: 010-2000-2000

kids:
  - age: 15
    name: 'kim younghee'
  - age: 12
    name: 'kim youngsu'
```

[XML]

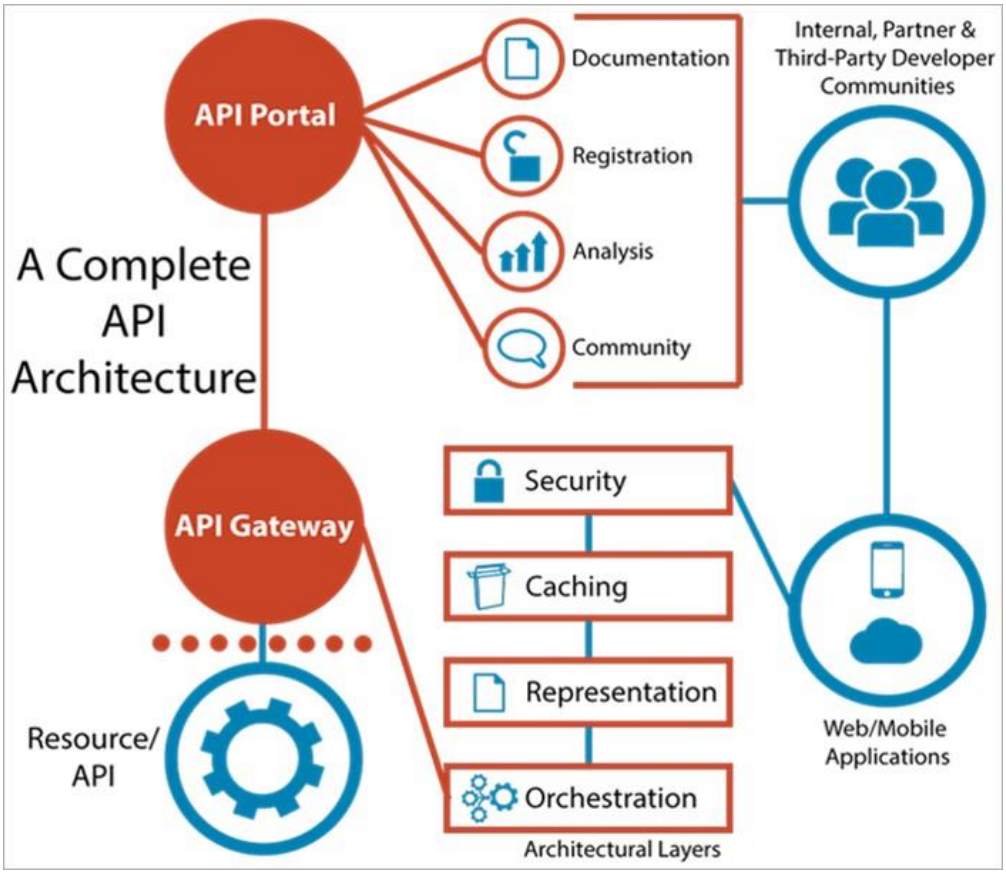
```xml
<root>
  <phone_number>
    <office>02-300-3000</office>
  </phone_number>
  <phone_number>
    <home>02-200-2000</home>
  </phone_number>
  <phone_number>
    <mobile>010-2000-2000</mobile>
  </phone_number>
  <kids>
    <age>15</age>
    <name>kim younghee</name>
  </kids>
  <kids>
    <age>12</age>
    <name>kim youngsu</name>
  </kids>
</root>
```

8. 오픈 플랫폼의 구성요소

오픈 API 플랫폼은, 오픈 API 가 배포되어 요청/응답 처리하는 front-end 영역과 API 서비스 로직이 실행되고 back-end 시스템과 연계하는 midlle-end 영역으로 나눌 수 있다.

APIM(API Management)

오픈 플랫폼의 Front-end 에 위치하여 API 사용자로부터의 API 요청을 처리하는 API Gateway 와 API 이해관계자들간의 의사소통 창구의 역할을 API Portal 로 구성되며, 이렇게 만들어진 오픈 API 를 별도의 오픈 API Store 에 판매할 수도 있다.



[그림 1] APIM 아키텍처 구성요소 / 출처: www.apiacademy.co

API Gateway

API 처리 요청에 대한 Proxy 로서 API 서버(혹은 AP 서버) 호출 대행 접점이다. API 처리를 위한일종의 고도화된 웹서버 인 셈이다. 따라서 웹서버를 굳이 두지 않아도 되고 회사의 아키텍처 표준이 있을 경우 웹서버를 앞 단에 둘 수도 있으나 이 경우 API Gateway 솔루션에서 제공하는 보안, 성능 처리 등의 유용한 기능 일부를 제한 받을 수 있다. 그렇다면 API Gateway 의 기능은 어떤 것들이 있을까?

1. 보안

- API 호출 제어: 북한, 중국 등 서비스 대상 외 국가 혹은 부정업체로부터의 호출을 제외하거나(Blacklist 방식), 계약된 업체나 사용자로부터만 호출을 허용하는 방식(Whitelist 방식)이 있다.

- API 인증: API Key 방식, OAuth2.0 인증 등을 통한 API 사용 인증 처리로써 API Portal 에서 발급받은 Key, Credential 등을 활용한다.

- API 전송 데이터 위변조 방지: 클라이언트와 서버 간 상호 통신 중 해커에 의해 중요한 파라미터를(예:

계좌번호) 위변조 하여 부정 사용되는 것을 막기 위한 처리로써 파라미터와 함께 전송하는 해쉬코드를 통하여 확인한다.

- SQL Injection, XSS, CSRF, XML 방화벽 등 웹 어플리케이션 방화벽 기능

2. 성능

- **지능형 부하분산:** 백엔드의 복수의 API 서버에 트랜잭션을 분산 처리하여 동시 처리율을 높일 수 있는 설정 선택 가능하다. 또한, API 서버 등의 장애 발생 시 자동 fail-over 처리, 관리자에 의한 설정으로 fail-back 처리 – 제외된 API 서버의 재참여 – 설정 등이 가능하다(가용성).

- **유량제어:** API 별로 호출할 수 있는 최대 횟수, 동시 호출 개수 등을 설정하여 특정 API 의 과도 사용으로 인한 시스템 자원의 고갈 및 타 사용자 및 API 의 성능에 영향을 주지 않도록 조절할 수 있다. 어플리케이션 레벨의 QoS(Quality of Service) 라고 볼 수 있다.
- **캐싱:** 반복된 조회 API 의 값을 캐싱하여 처리 성능을 향상

- **API Orchestration:** 다수의 API 를 조합하여 새로운 복한 API 조합 가능하나, 단순한 조합이 아니라면 API 서버에서 기능을 구현하는 것이 복잡한 룰을 처리하고 모니터링 하기 용이하다.

API Portal

API 소비자와 제공자가 소통할 수 있는 통로이며, API 제공자는 API portal 을 통하여 API Gateway 에 인증 Key, 사용기관 등을 설정할 수 있다. API 는 일반적으로 프로그램에서 호출되므로 개발자들이 API 사용 허가를 요청하고, 서버에서 제공하는 API 스펙을 확인하고 간결하게 사전 테스트 해볼 수 있는 기능이 제공되는 공간이다. 또한 API 사용 현황에 대한 간단한 모니터링, 기간 별 과금 정보를 확인하고 API 제공자와 조율할 수 있다.

- **Documentation:** API 목록, 사양(Specification)을 등록, 검색, 확인할 수 있다. 또한, 샘플 데이터를 통해서 간단히 API 동작을 테스트하고 피드백을 남길 수 있다. 이를 위하여 앞서 살펴보았던 RAML, OAS 3.0, Swagger 등을 활용한다.

- **Registration:** API 사용자, 관리자 등을 역할 기반으로 등록하고 사용기관에서 자체적으로 관리할 수 있는 기능을 제공한다(접근제어). API Key, Credential 을 요청, 발급받고 관리가 가능하다.
- **Analysis:** API 사용 현황을 API 사용자 측에서 확인할 수 있는 그래프 혹은 표 등을 제공하며, API 사용에 대한 과금 정책 및 기간 별 사용 현황 분석 및 금액을 확인할 수 있다.

- **Community:** API 생태계를 지원하기 위하여 API 관련 개발자 들 간의 소통 통로로서, FAQ, Q&A 질의 대응, 신규 API 디자인 등을 위한 개발자 커뮤니티 게시판 등을 제공한다.

API Platform 으로의 확장

API Portal, API Gateway 만 있으면 오픈 API 플랫폼이라고 할 수 있을까? APIM 은 단지 외부와 연결할 수 있는 기본적인 통로일 뿐이다. 오픈 플랫폼이란 여러 이해관계자가 다방향으로 소통하고 새로운 서비스가 창출될 수 있어야 할 것이다. 이를 위해서 서비스 융합(convergence), 서비스 조합(orchestration)을 할 수 있는 레이어들이 필요하다.

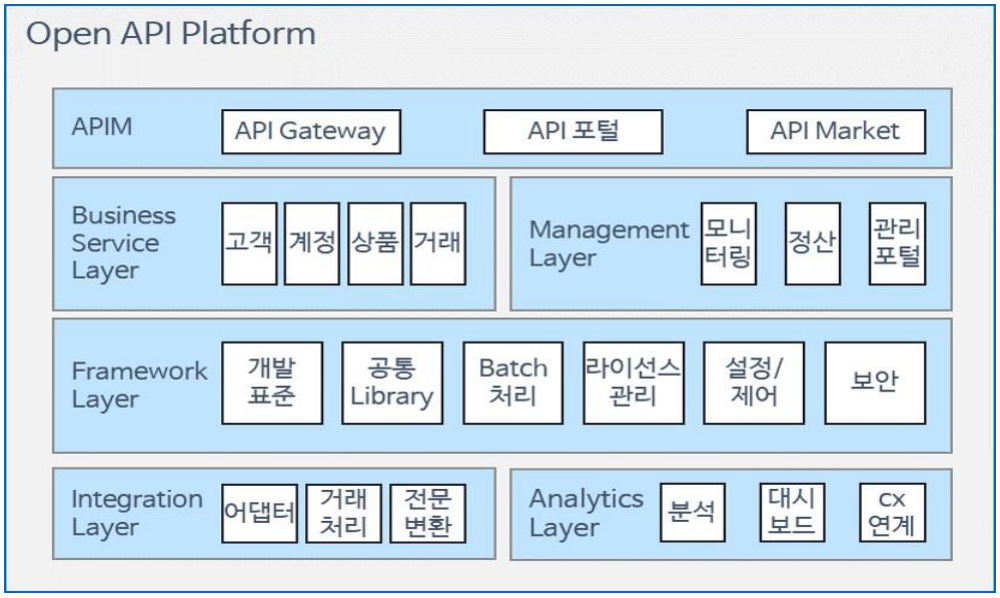
- **Business Service Layer:** 금융, 유통, 물류, 제조, 통신 등 업계에서 활용되는 통상적인 비즈니스 기본 컴포넌트(예: 은행의 경우 계좌관리)를 조작할 수 있는 기능과 이를 활용한 핀테크 업체와의 융합 서비스 컴포넌트(예: P2P 투자자/대출자관리)의 서비스 흐름을 구현할 수 있는 어플리케이션, 데이터 처리 레이어이다. 일반적으로 API 서버라고 불리우며 이 레이어와 이를 실행하고 관리할 수 있는 Framework Layer 가 있어야 다양한 서비스 융복합을 할 수 있다.

- **Management Layer:** API 사용자에 대한 빌링, 정산, 플랫폼 모니터링 및 관리 포털 기능 등을 제공한다.

- **Framework Layer:** API 의 서비스 로직을 프로그래밍하고 배포, 온라인/배치 실행, 개발표준 관리, 공통 라이브러리, 플랫폼 보안 기능 등을 제공하는 플랫폼 코어 프레임워크 영역이다.

- **Integration Layer:** 플랫폼 제공자의 Back-end 시스템과 다양한 통신 방식으로 거래 연계, 전문변환, 모니터링 한다.

- **Analytics Layer:** 플랫폼에 축적된 다양한 데이터를 분석, 시각화 하고 CX 에 활용할 수 있도록 데이터를 가공하여 제공한다.



[그림 2] 오픈 API Platform 아키텍처 예시

플랫폼 성격에 따라 기능을 세분화 혹은 통합하여 Layer 를 조정하거나 선별 적용할 수 있다.

오픈 API 플랫폼 고려사항

1. 중소 규모의 핀테크 업체와 비용 절감

중소 규모의 핀테크 업체와 서비스 개발을 하고 플랫폼을 통하여 이윤을 창출하려면 일종의 박리다매 형태로 플랫폼이 운영되는 경우가 많다. 즉, 고가의 솔루션, 인프라를 투입하기에는 채산성이 맞지 않기 때문에, 오픈소스 솔루션을 적극 도입하여 구매 및 유지보수 비용을 대폭 없애고, 워크로드에 유연한 인프라 가상화, 퍼블릭 클라우드를 활용하여 플랫폼 수요가 폭발적이거나 외면당했을 경우에 투자 비용을 절감할 수 있도록 플랫폼을 구축해야 한다. 오픈소스 기술 전문가의 육성, 확보가 쉽지 않은 과제이긴 하다.

2. platform 보안 개인정보

오픈 플랫폼 서비스는 기존 다른 곳에서 이미 제공되고 있는 서비스가 아닌 경우가 대부분이기 때문에, 플랫폼 참여업체의 개인정보 보유, 활용에 따른 정보보안 법, 규정에 위배되지 않는지 검토하고 이에 대한 대책을 적용하는 것이 중요하다. 다수의 참여자를 연결하는 오픈 플랫폼의 특성상, 보안이 뚫리면 플랫폼 내부뿐만 아니라 연결된 시스템으로까지 보안침해사고가 전파되기 용이하므로, 보안 가이드를

세밀하게 정립하고 참여자와의 계약 형태에 따라 이를 준수할 것을 요구하고 상시 감시하는 활동이 필요하다.

3. 플랫폼 운영 거버넌스, 인력 확보

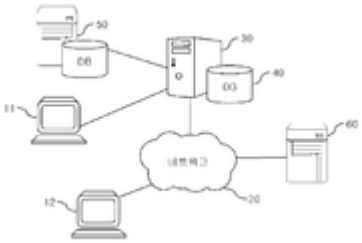
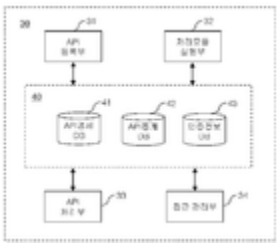
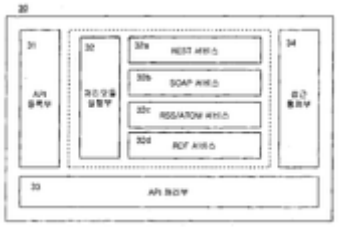
오픈 API 플랫폼은 특성상 자체 완결형이 아니라 수많은 다양한 시스템과의 연계, 서비스 조합이 필요한 공간이다. 하지만 지향하는 플랫폼 비즈니스과는 무관하게 기존의 시스템 운영 방식과 유사하게 조직, 인력 구성을 하는 경향을 볼 수 있다. 디지털 트랜스포메이션의 비전에 따라 플랫폼의 방향성은 정해지지만 외부 참여자에 따라 플랫폼 비즈니스는 어떻게 확장되어 갈지는 미리 규정하기 힘들다. 세상의 의미있는 서비스를 발굴하여 자신의 비즈니스를 충돌시키고 새로운 플랫폼 서비스를 재창조하는 비즈니스 DMZ(demilitarized zone) 영역인 것이다. 열린 사고와 잡다할 정도로 다양한 비즈니스, 기술 경험을 가진 인력과 이를 뒷받침 할 수 있는 플랫폼 거버넌스가 필수적이라 하겠다.

4. 오픈 API 기반 서비스 인터페이스 제공 시스템 및 방법

Abstract

콘텐츠 서버에서 제공하는 콘텐츠 서비스를 API(Application Programming Interface)로 등록하여 제공하고 실행시켜주는 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법에 있어서, 콘텐츠 서비스를 규격화된 오픈 API로 등록하고, 등록된 콘텐츠 서비스(이하 서비스 API)에 대한 서비스 명세를 작성하는 API 등록부; 오픈 API의 처리 모듈을 실행시키는 처리모듈 실행부; 및, 웹사이트 서버의 사용자로부터 서비스 API의 실행 요청을 수신하면, 오픈 API 처리 모듈에 의해 서비스 API를 실행시켜주는 API 처리부를 포함하는 구성을 마련한다. 상기와 같은 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법에 의하여, 콘텐츠 서버의 관리자는 별도의 어플리케이션 개발을 하지 않고도 콘텐츠를 공유하는 서비스를 타 웹사이트 서버에서 제공할 수 있다.

Images (23)







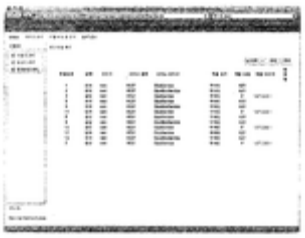
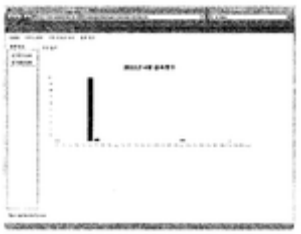
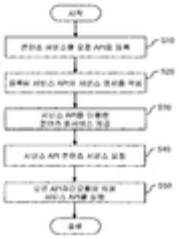










Description

오픈 ＡＰＩ 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법

본 발명은 콘텐츠 서버에서 제공하는 콘텐츠 서비스를 API(Application Programming Interface)로 등록하여 제공하고 실행시켜주는 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법에 관한 것이다.

일반적으로 Open API(Application Programming Interface)란 응용프로그램(Application Program)을 개발할 때, 타 시스템이 특정 기능을 호출해서 활용할 수 있도록 하는 접속 규약을 말한다. 즉, Open API 는 불특정 다수의 사용자가 응용프로그램을 쉽고 용이하게 개발 활용할 수 있도록 API 를 외부에 공개하는 것이다.

이때, 당사자 간 규약, 즉, 불특정 다수가 사용이 가능하도록 접속 규약은 표준화 되어야 한다. 웹서비스(Web Services)가 국제 표준화된 대표적인 Open API 방식이라 할 수 있다.

상기와 같은 오픈 API 를 통해 인터넷 상의 웹 서비스를 서비스 단위별로 제공하여 공유할 수 있는 공유 서비스가 가능하다. 즉, 공유 서비스란 이용자와 제공자의 정보시스템 환경이 서로 달라도 연결가능 하도록 표준방식(표준인터페이스)을 사용하여 외부에서 호출이 가능한 업무모듈을 말한다.

공유 서비스를 통해 서비스 지향 아키텍쳐(Service Oriented Architecture)로 웹 서비스의 설계를 할 수 있다. 서비스 지향 아키텍처는 분산된 자원들(정보나 업무기능, 시스템 등)을 '서비스화'하여 '연결'하고 '활용'하는 새로운 기술과 비즈니스 패러다임이다.

서비스 지향 아키텍처는 표준을 활용하는 상호운용성이 높은 구조를 제공하므로, 자원들의 운영 환경이 이질적이어도 연결과 활용이 쉽고, 변화대응력이 높다는 장점을 가지고 있다. 또한, 지식정보사회 실현을 위해 서비스지향아키텍처에 기반하여 공유서비스와 인프라를 구축하고 있는 것이 필요하다.

즉, 서비스 지향 아키텍처를 통해, 서비스 제공자들은 이용자 환경에 무관한(미래의 이용자 포함) 정보시스템 개발과 제공이 가능하다. 또, 서비스 이용자들은 다양한 기관이나 부서들로부터 제공되는 서비스들을 별도 통합작업 또는 제공자들의 환경별 변환작업 없이도 업무 프로세스를 구성하거나 변형하는데 활용할 수 있다. 특히, 다양한 단말과 네트워크들도 「공유 서비스화」를 통하여 최소 비용으로 다양한 응용사업이나 비즈니스에서 활용될 수 있다.

특히, 공공기관이나 민간 기업에서 높은 사회/경제적 가치를 지닌 자원(콘텐츠)의 종류와 양이 지속적으로 증가함에 따라, 자원의 공유와 활용에 대한 수요가 많아지고 있다.

그러나 이를 쉽고 효과적으로 관리할 수 있는 개발 도구(tool)가 부재하여 공유서비스 작성시 별도의 어플리케이션을 개발하여야 한다. 즉, 종래 시스템에서 공유서비스를 생산하기 위해서는 어플리케이션 개발자가 SOAP(Simple Object Access Protocol), REST(Representational State Transfer)등의 서비스 방식을 선택하여 별도의 개발이 필요하였고, 그에 따른 비용과 기간이 필요하였다. 이로 인해, 공유 서비스화 과정에서 비효율성이 존재하고 있다.

따라서 이를 극복하기위해 공유서비스의 생산 및 관리를 보다 편리하고 별도의 추가 비용 없이도 공유서비스를 생산하고 관리할 수 있는 기술이 절실하다.

본 발명의 목적은 상술한 바와 같은 문제점을 해결하기 위한 것으로, 콘텐츠 서버에서 제공하는 콘텐츠 서비스를 API(Application Programming Interface)로 등록하여 제공하고 실행시켜주는 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법을 제공하는 것이다.

상기 목적을 달성하기 위해 본 발명은 콘텐츠 서버에서 제공하는 콘텐츠 서비스를 API(Application Programming Interface)로 등록하여 제공하고 실행시켜주는 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 관한 것으로서, 상기 콘텐츠 서비스를 규격화된 오픈 API 로 등록하고, 등록된 콘텐츠 서비스(이하 서비스 API)에 대한 서비스 명세를 작성하는 API 등록부; 상기 오픈 API 의 처리 모듈을 실행시키는 처리모듈 실행부; 및, 웹사이트 서버의 사용자로부터 서비스 API 의 실행 요청을 수신하면, 상기 오픈 API 처리 모듈에 의해 상기 서비스 API 를 실행시켜주는 API 처리부를 포함하는 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 시스템은, 웹사이트 서버의 사용자로부터 상기 서비스 API 의 실행 요청을 수신하면, 상기 서비스 API 의 실행을 요청하는 사용자를 인증하고 상기 사용자에 의한 접근 통제를 수행하는 접근 관리부를 더 포함하는 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 접근 관리부는 사용자키(AccessKey)와 사용자비밀키(AccessSecretKey)를 이용하는 키기반 인증 및 IP 주소(Address)를 이용하는 IP 주소 인증 중 어느 하나 이상을 이용하는 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 접근 관리부는 상기 서비스 API 의 실행에 관한 통계정보를 수집하여 저장하는 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 API 처리부는 상기 서비스 API 를 실행한 결과를 웹 프로토콜에 의해 웹 데이터 형식으로 전환하여 상기 웹사이트 서버에 전송하는 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 웹 프로토콜은 HTTP(hypertext transfer protocol]) 1.0 이상의 프로토콜이고, 상기 웹 데이터 형식은 XML(eXtensible Markup Languag) 또는 JSON(JavaScript Object Notation) 인 것을 특징으로 한다.

또, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템에 있어서, 상기 오픈 API 는 REST(Representational State Transfer), SOAP(Simple object access, Protocol), RSS/ATOM, RDF(Resource Description Framework) 중 어느 하나 이상인 것을 특징으로 한다.

또한, 콘텐츠 서버에서 제공하는 콘텐츠 서비스를 API(Application Programming Interface)로 등록하여 제공하고 실행시켜주는 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 방법에 관한 것으로서, (a) 상기 콘텐츠 서비스를 규격화된 오픈 API 로 등록하는 단계; (b) 등록된 콘텐츠

서비스(이하 서비스 API)에 대한 서비스 명세를 작성하는 단계; (c) 웹사이트 서버가 상기 서비스 명세에 따라 상기 서비스 API 를 이용한 콘텐츠 서비스를 제공하는 단계; (d) 사용자 단말에서 상기 웹사이트 서버에 접속하여 상기 서비스 API 의 콘텐츠 서비스를 요청하는 단계; (e) 웹사이트 서버에서 사용자 단말로부터 서비스 API 의 실행 요청을 수신하면, 상기 오픈 API 처리 모듈에 의해 상기 서비스 API 를 실행시켜주는 단계를 포함하는 것을 특징으로 한다.

또한, 상기 방법을 수행하는 프로그램을 기록한 컴퓨터로 읽을 수 있는 기록매체에 관한 것이다.

상술한 바와 같이, 본 발명에 따른 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법에 의하면, 콘텐츠 서버의 관리자는 별도의 어플리케이션 개발을 하지 않고도 콘텐츠를 공유하는 서비스를 타 웹사이트 서버에서 제공할 수 있는 효과가 얻어진다.

특히, 오픈 API 기반 콘텐츠 서비스 인터페이스 제공 시스템 및 방법에 의하면, 서비스 담당자가 쉬운 GUI 툴을 사용하여 SOAP, REST, RSS, ATOM, RDF 등의 방식으로 공유서비스를 생산 및 수정, 삭제, 배포함으로써, 별도의 비용이 소요되지 않고 개발기간도 단축 수 있는 효과가 얻어진다.